all-pairs Bellman-Ford (1)

one algorithm to find all shortest paths in graph (network)

$d(A, B) = \text{best distance from A to B}$

$p(A, B) = \text{next node on path from A to B}$

initially $d(X, Y) = \infty$ for all nodes X, Y

repeatedly* do the following:

for each link: (A to B, distance c):

    for each node X:

    if $c + d(B, X) < d(A, X)$,

    then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

all-pairs Bellman-Ford (2)

for each link (A to B, distance c):

for each node X:

if $c + d(B, X) < d(A, X)$,

then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

if A$\rightarrow$B$\rightarrow$X faster than current A$\rightarrow$X route
then choose it over current route

when choosing: update distance A$\rightarrow$X + next link for A$\rightarrow$X

all-pairs Bellman-Ford (2)

for each link (A to B, distance c):

for each node X:

if $c + d(B, X) < d(A, X)$,

then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

if A⟶B⟶X faster than current A⟶X route

then choose it over current route

when choosing: update distance A⟶X + next link for A⟶X

all-pairs Bellman-Ford (2)

for each link (A to B, distance c):

for each node X:

if $c + d(B, X) < d(A, X)$,

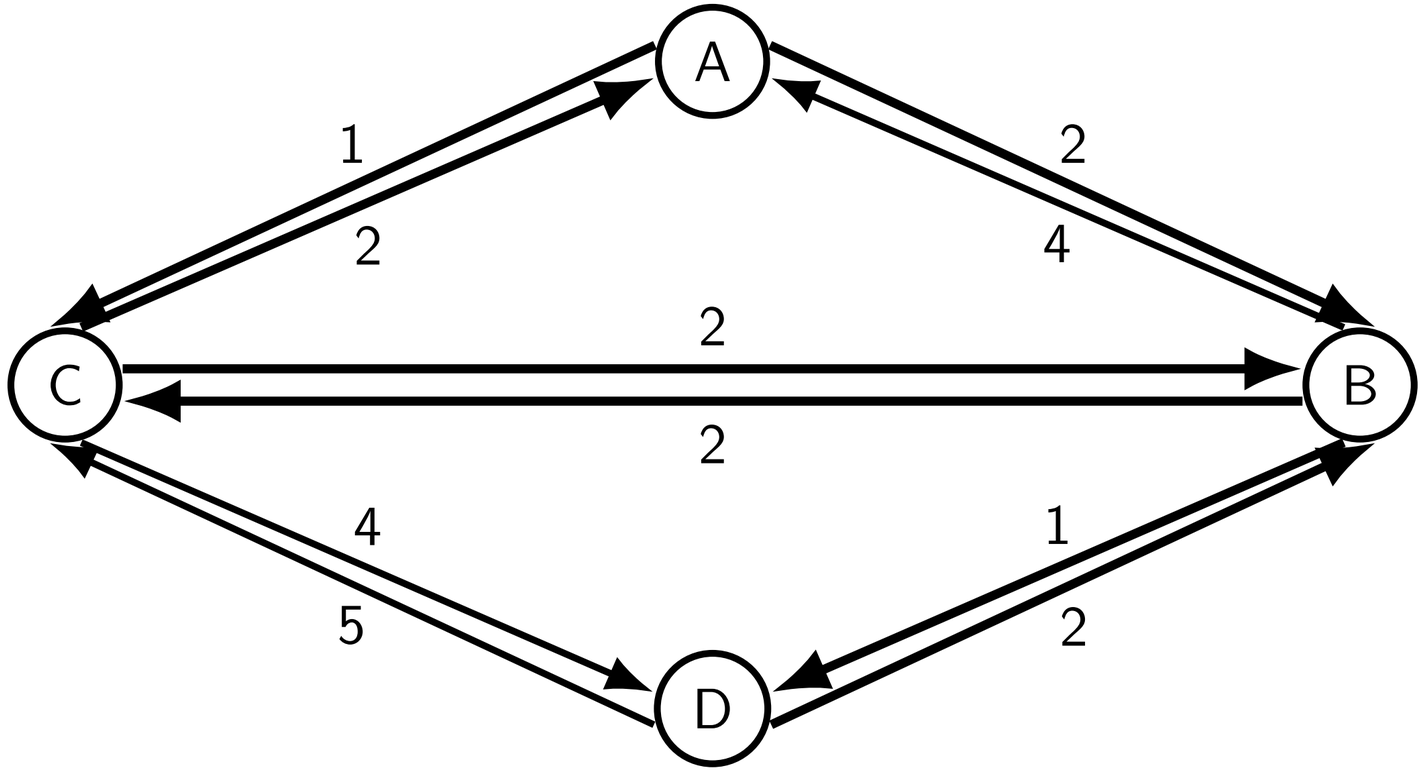
then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

if A$\rightarrow$B$\rightarrow$X faster than current A$\rightarrow$X route

then choose it over current route

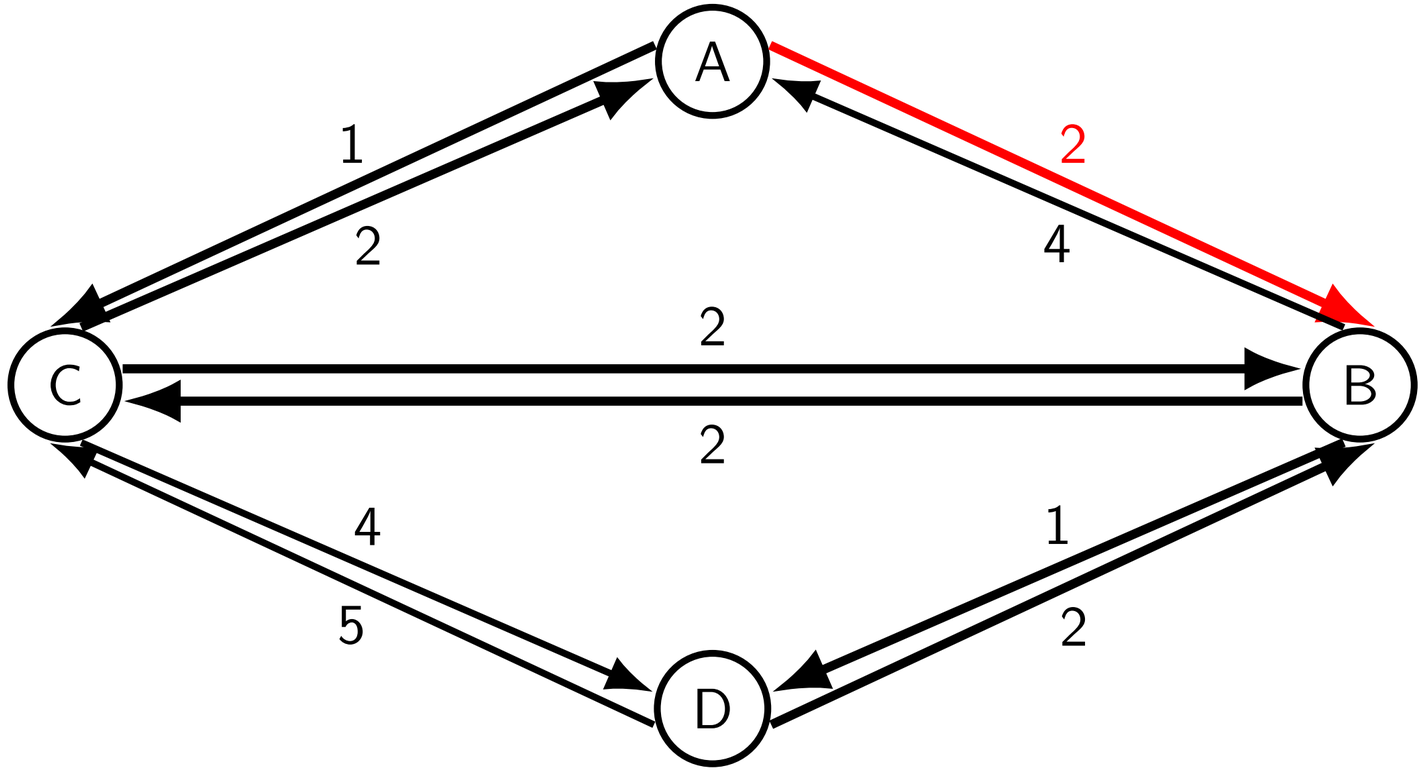
when choosing: update distance A$\rightarrow$X + next link for A$\rightarrow$X

running Bellman-Ford



	A	B	C	D
A	0/A	∞/—	∞/—	∞/—
B	∞/—	0/B	∞/—	∞/—
C	∞/—	∞/—	0/C	∞/—
D	∞/—	∞/—	∞/—	0/D

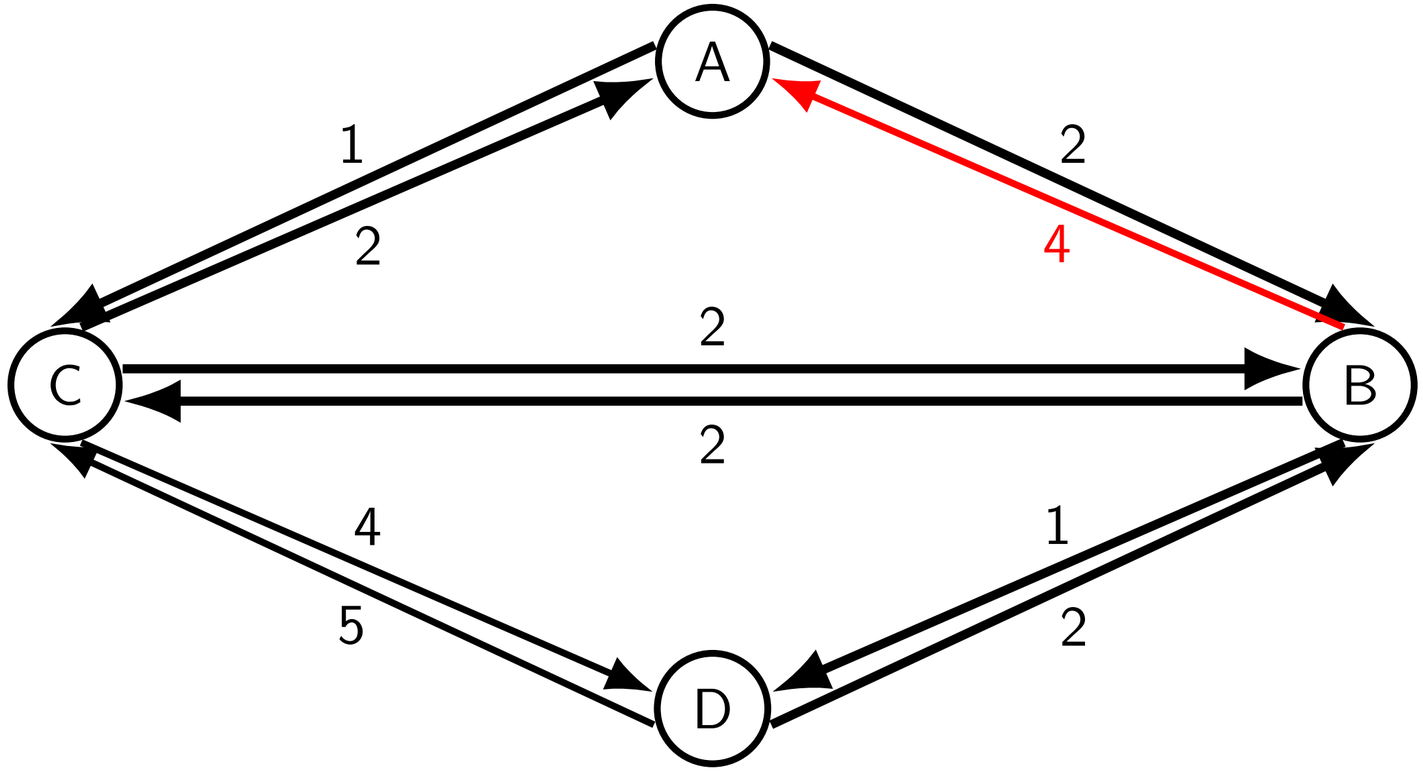
running Bellman-Ford



	A	B	C	D
A	0/A	*2/B*	∞/—	∞/—
B	∞/—	0/B	∞/—	∞/—
C	∞/—	∞/—	0/C	∞/—
D	∞/—	∞/—	∞/—	0/D

(route B to B, distance 0) + (link A to B, distance 2) =
route B to A, distance 2+0=2 via B

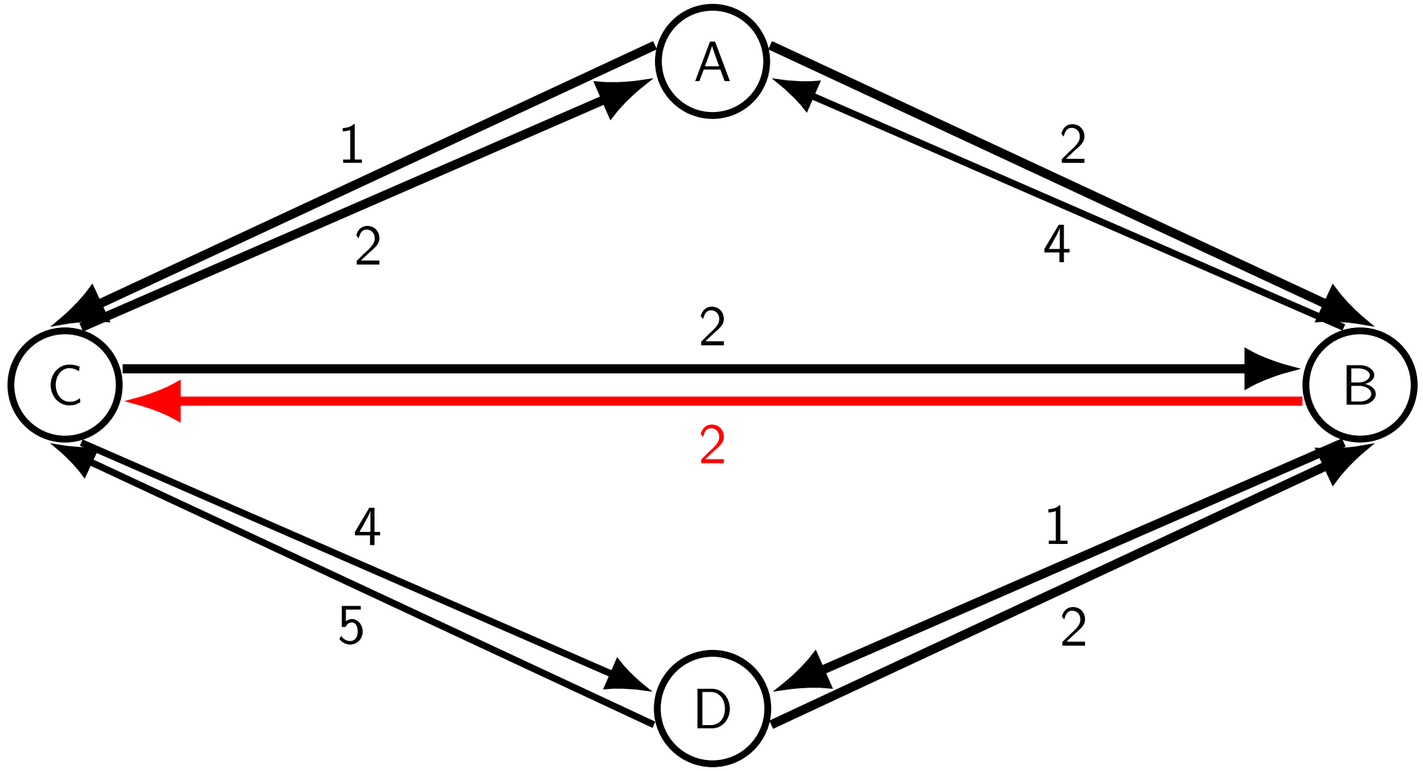
running Bellman-Ford



	A	B	C	D
A	0/A	2/B	∞/—	∞/—
B	*4/A*	0/B	∞/—	∞/—
C	∞/—	∞/—	0/C	∞/—
D	∞/—	∞/—	∞/—	0/D

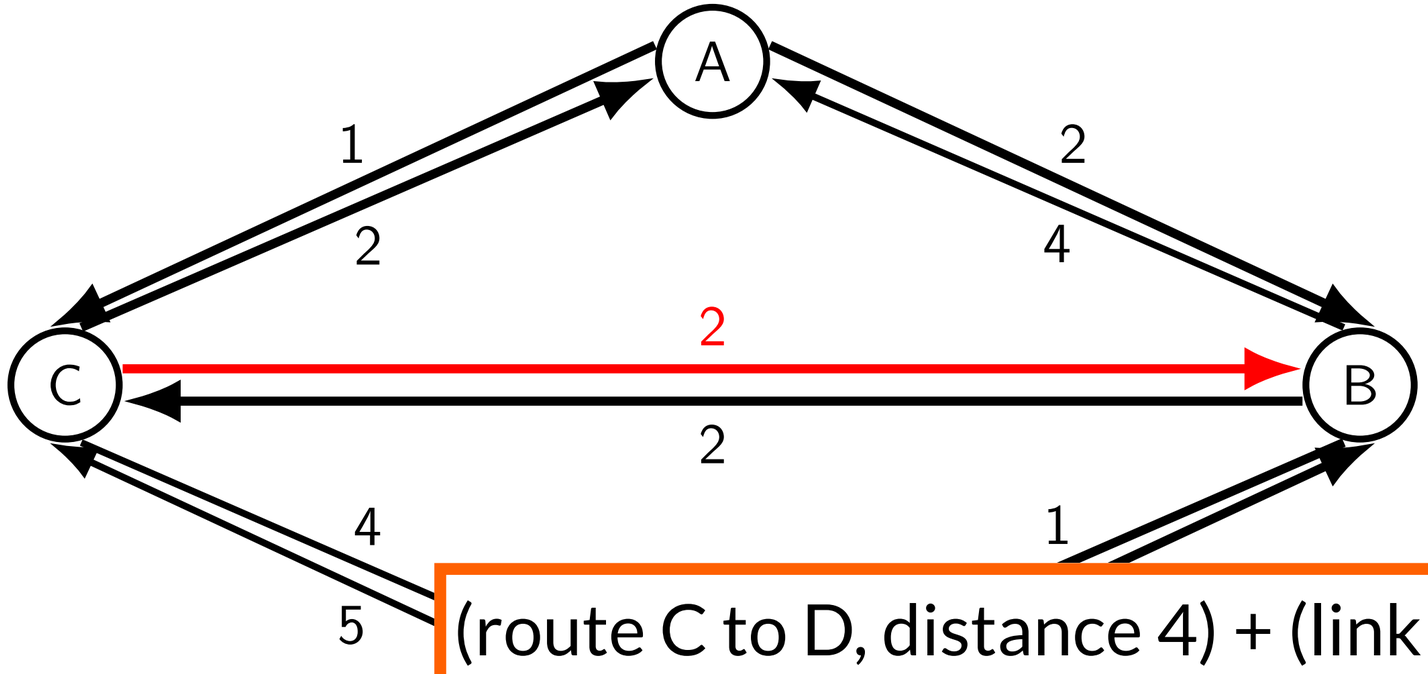
(route A to A, distance 0) + (link B to A, distance 4) =
route A to B, distance 4+0=4 via A

running Bellman-Ford



	A	B	C	D
A	0/A	2/B	∞/—	∞/—
B	4/A	0/B	*2/C*	∞/—
C	∞/—	∞/—	0/C	∞/—
D	∞/—	∞/—	∞/—	0/D

running Bellman-Ford

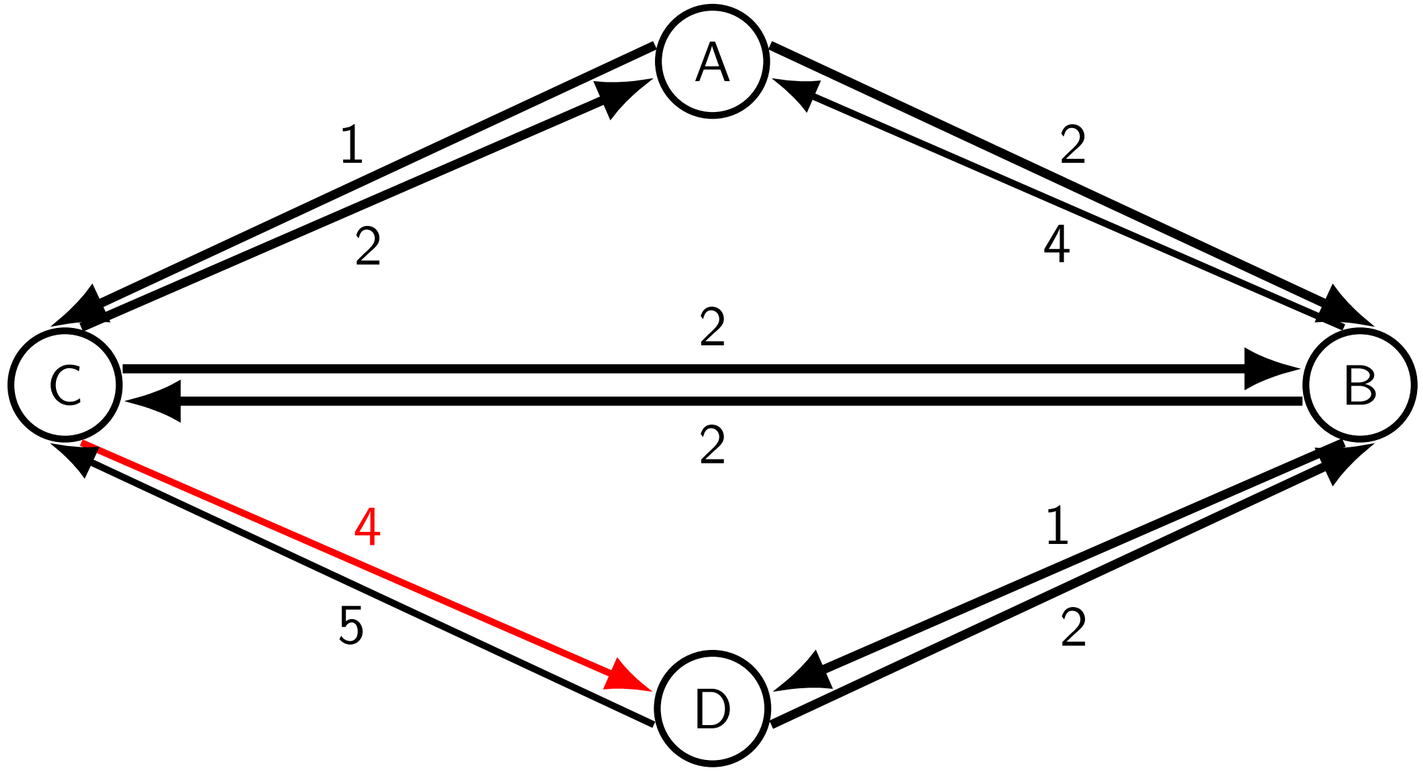


	A	B	C	D
A	0/A	2/B	∞/—	∞/—
B	4/A	0/B	2/C	∞/—
C	*6/B*	*2/B*	0/C	∞/—
D	∞/—	∞/—	∞/—	0/D

(route C to D, distance 4) + (link A to C, distance 1) = route A to D, distance 4+1=5 via C

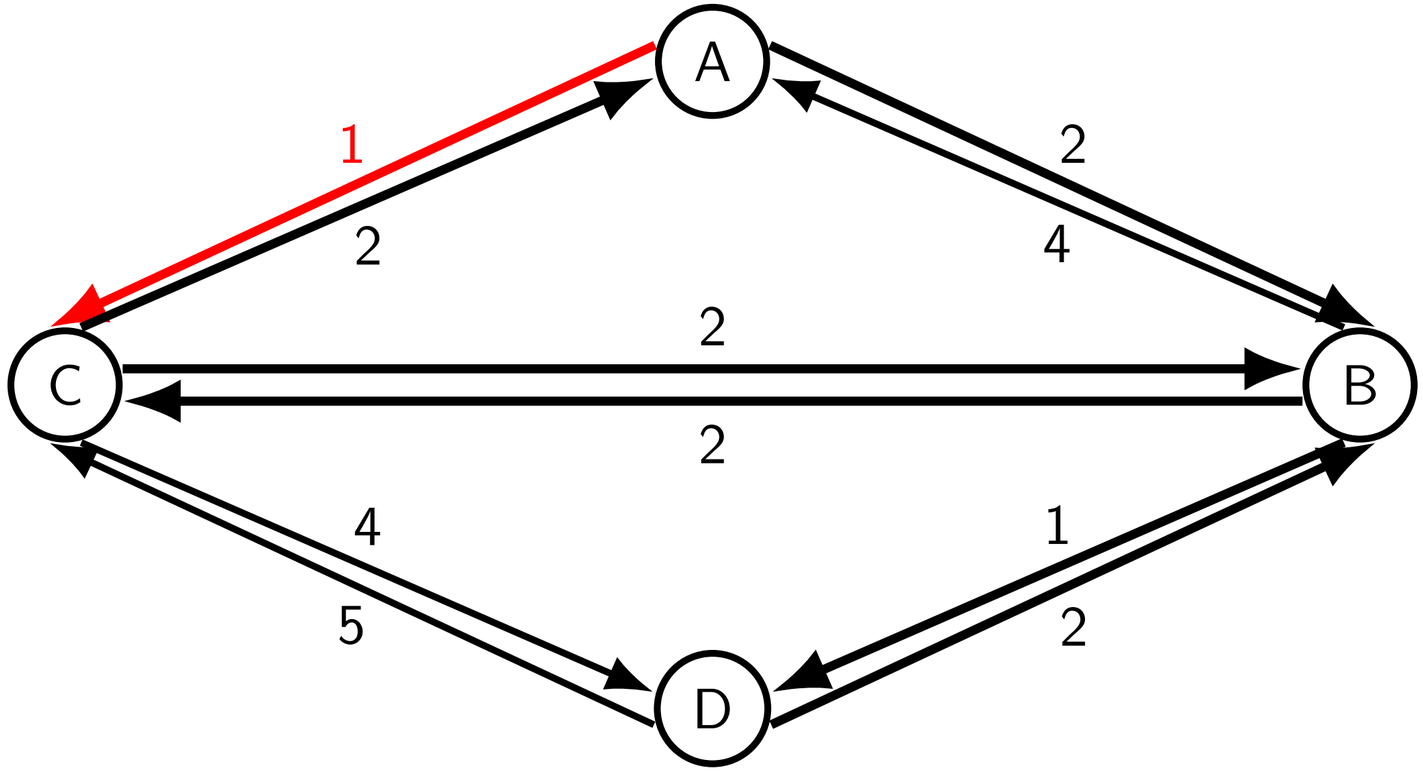
(route C to C, distance 0) + (link A to C, distance 1) = route A to C, distance 0+1=1 via C

running Bellman-Ford



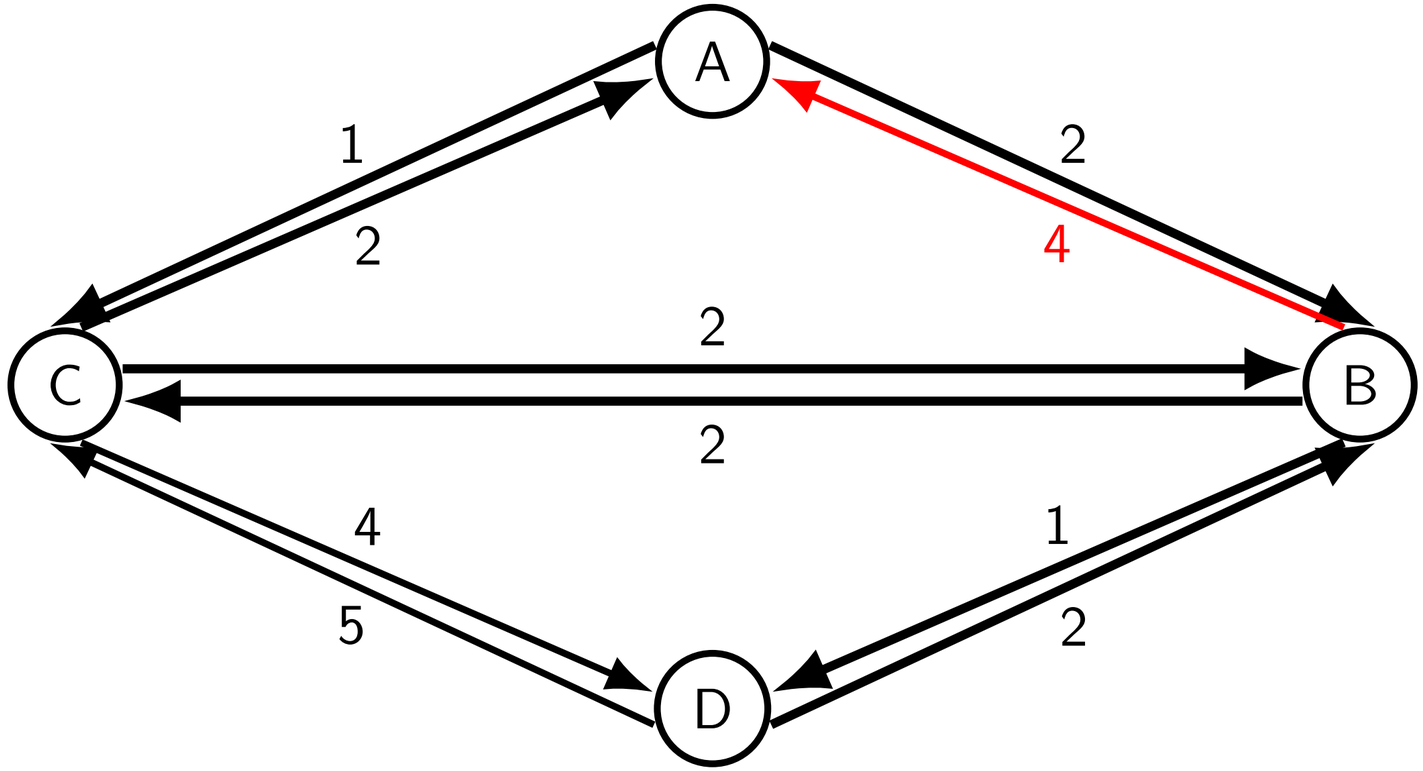
	A	B	C	D
A	0/A	2/B	∞/—	∞/—
B	4/A	0/B	2/C	∞/—
C	6/B	2/B	0/C	*4/D*
D	∞/—	∞/—	∞/—	0/D

running Bellman-Ford



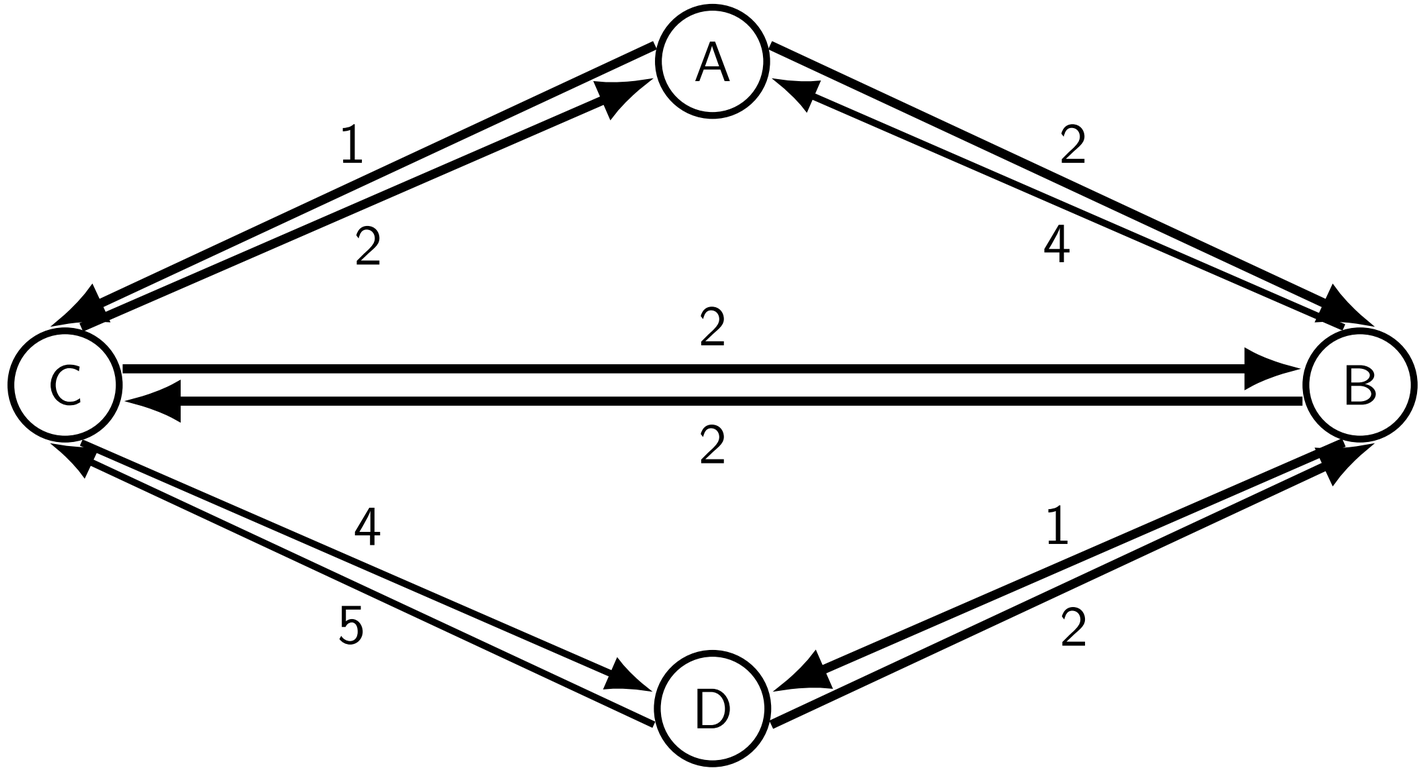
	A	B	C	D
A	0/A	2/B	*1/C*	*5/C*
B	4/A	0/B	2/C	∞/—
C	6/B	2/B	0/C	4/D
D	∞/—	∞/—	∞/—	0/D

running Bellman-Ford



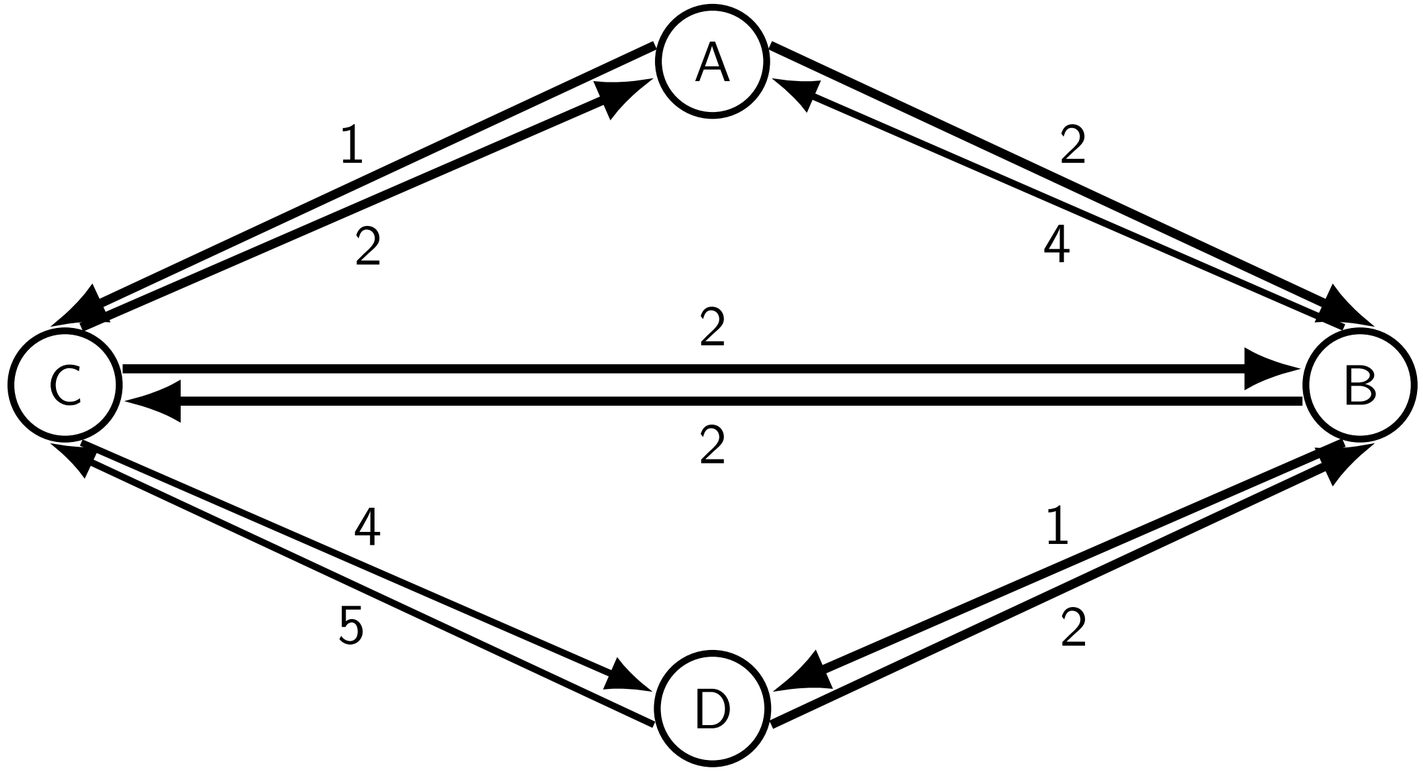
	A	B	C	D
A	0/A	2/B	1/C	5/C
B	4/A	0/B	2/C	*9/A*
C	6/B	2/B	0/C	4/D
D	∞/—	∞/—	∞/—	0/D

running Bellman-Ford



	A	B	C	D
A	0/A	2/B	1/C	5/C
B	4/A	0/B	2/C	9/A
C	6/B	2/B	0/C	4/D
D	∞/—	∞/—	∞/—	0/D

running Bellman-Ford



	A	B	C	D
A	0/A	2/B	1/C	3/B
B	4/A	0/B	2/C	1/D
C	2/A	2/B	0/C	4/D
D	6/B	2/B	4/B	0/D

distributing Bellman-Ford outline (1)

on each node A run:

repeatedly:

for each link *from A* (going to B, distance c)

for each node X:

if $c + d(B, X) < d(A, X)$,

then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

on node A: track $d(A, X), p(A, X)$

node A needs to get $d(B, X)$ to run algorithm

distributing Bellman-Ford outline (1)

on each node A run:

repeatedly:

for each link *from A* (going to B, distance c)

for each node X:

if $c + d(B, X) < d(A, X)$,

then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

on node A: track $d(A, X), p(A, X)$

node A needs to get $d(B, X)$ to run algorithm

distributing Bellman-Ford outline (1)

on each node A run:

repeatedly:

for each link *from A* (going to B, distance c)

for each node X:

if $c + d(B, X) < d(A, X)$,

then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

on node A: track $d(A, X), p(A, X)$

node A needs to get $d(B, X)$ to run algorithm

distributing Bellman-Ford outline (1)

on each node A run:

repeatedly:

for each link *from A* (going to B, distance c)

for each node X:

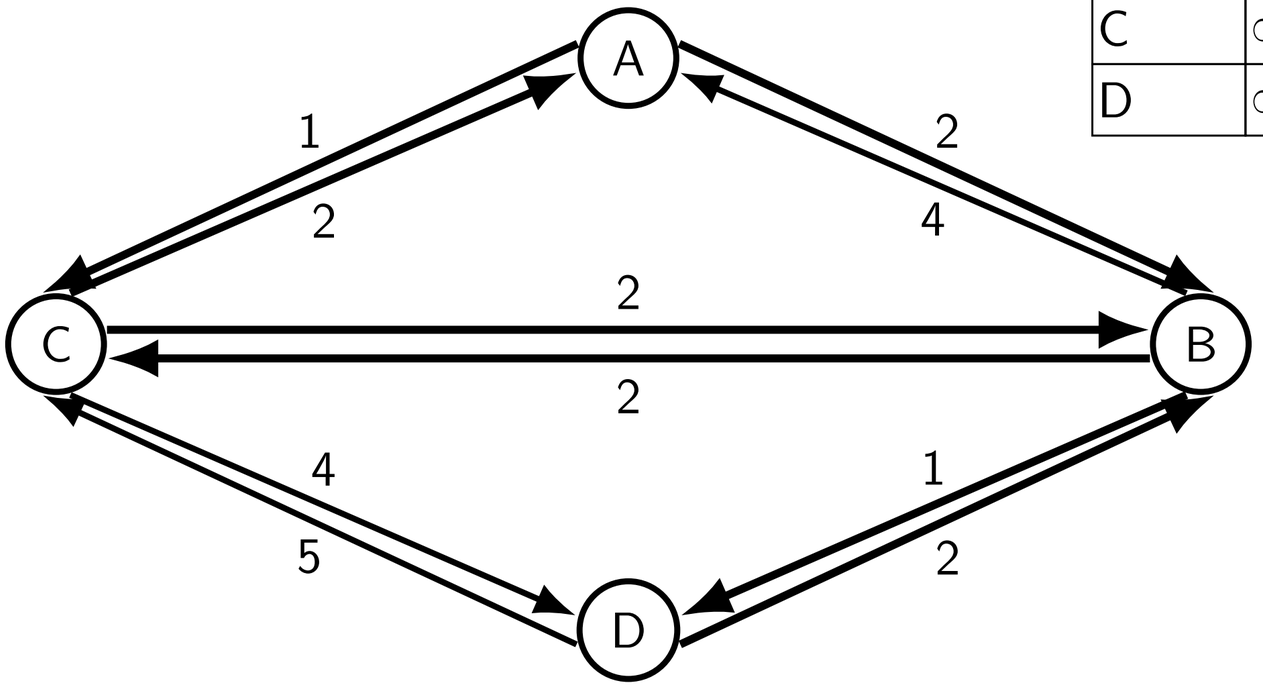
if $c + d(B, X) < d(A, X)$,

then $d(A, X) \leftarrow c + d(B, X), p(A, X) = B$

on node A: track $d(A, X), p(A, X)$

node A needs to get $d(B, X)$ to run algorithm

distributing Bellman-Ford



	A	B	C	D
A	0/A	∞/—	∞/—	∞/—
B	∞/—	0/B	∞/—	∞/—
C	∞/—	∞/—	0/C	∞/—
D	∞/—	∞/—	∞/—	0/D

distributing Bellman-Ford

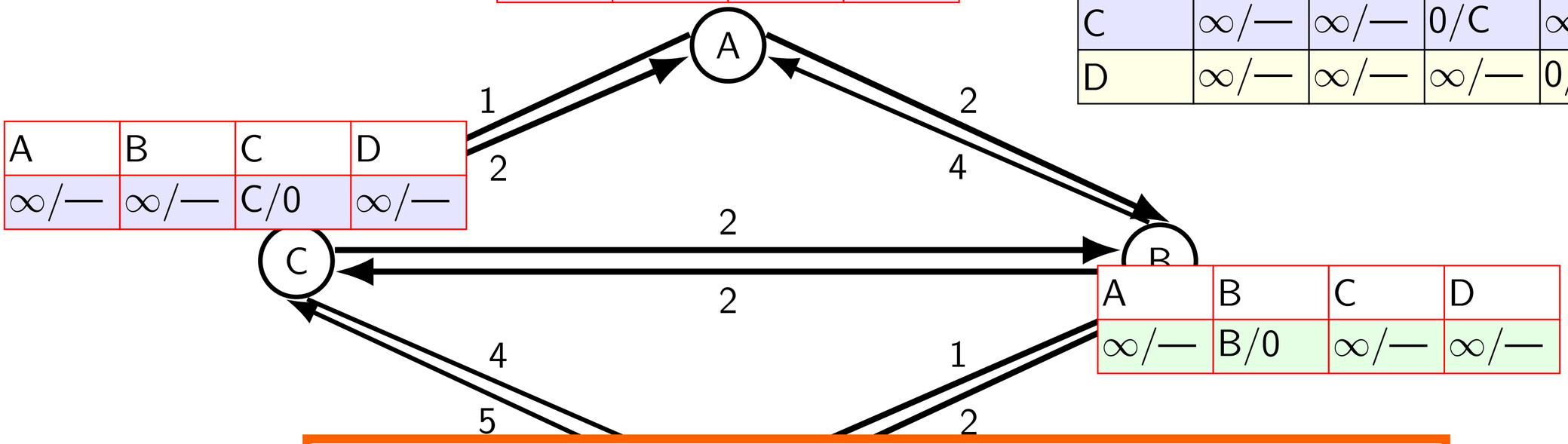
	A	B	C	D
A	0/A	∞/—	∞/—	∞/—
B	∞/—	0/B	∞/—	∞/—
C	∞/—	∞/—	0/C	∞/—
D	∞/—	∞/—	∞/—	0/D

A	B	C	D
A/0	∞/—	∞/—	∞/—

A	B	C	D
∞/—	∞/—	C/0	∞/—

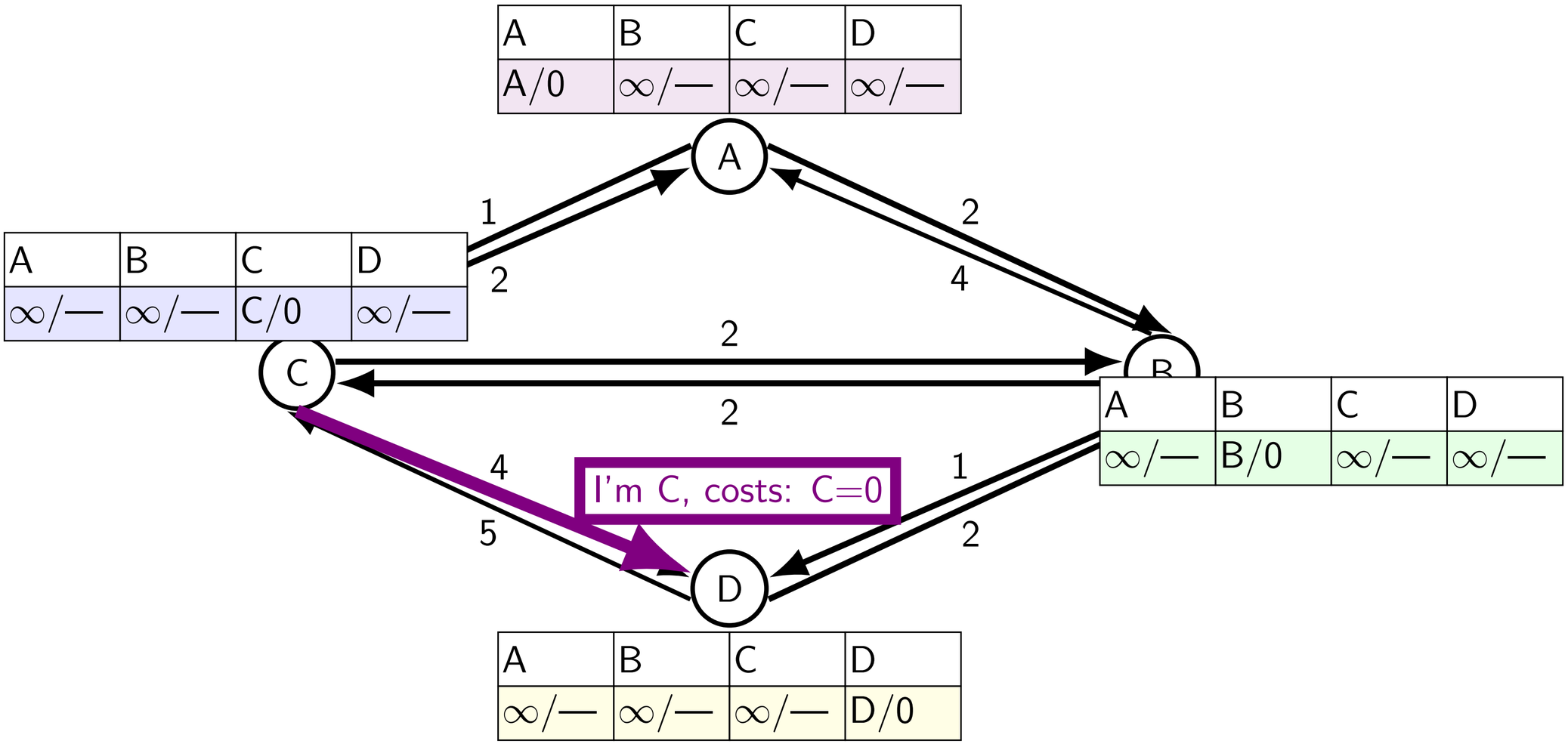
A	B	C	D
∞/—	B/0	∞/—	∞/—

A	B	C	D
∞/—	∞/—	∞/—	D/0

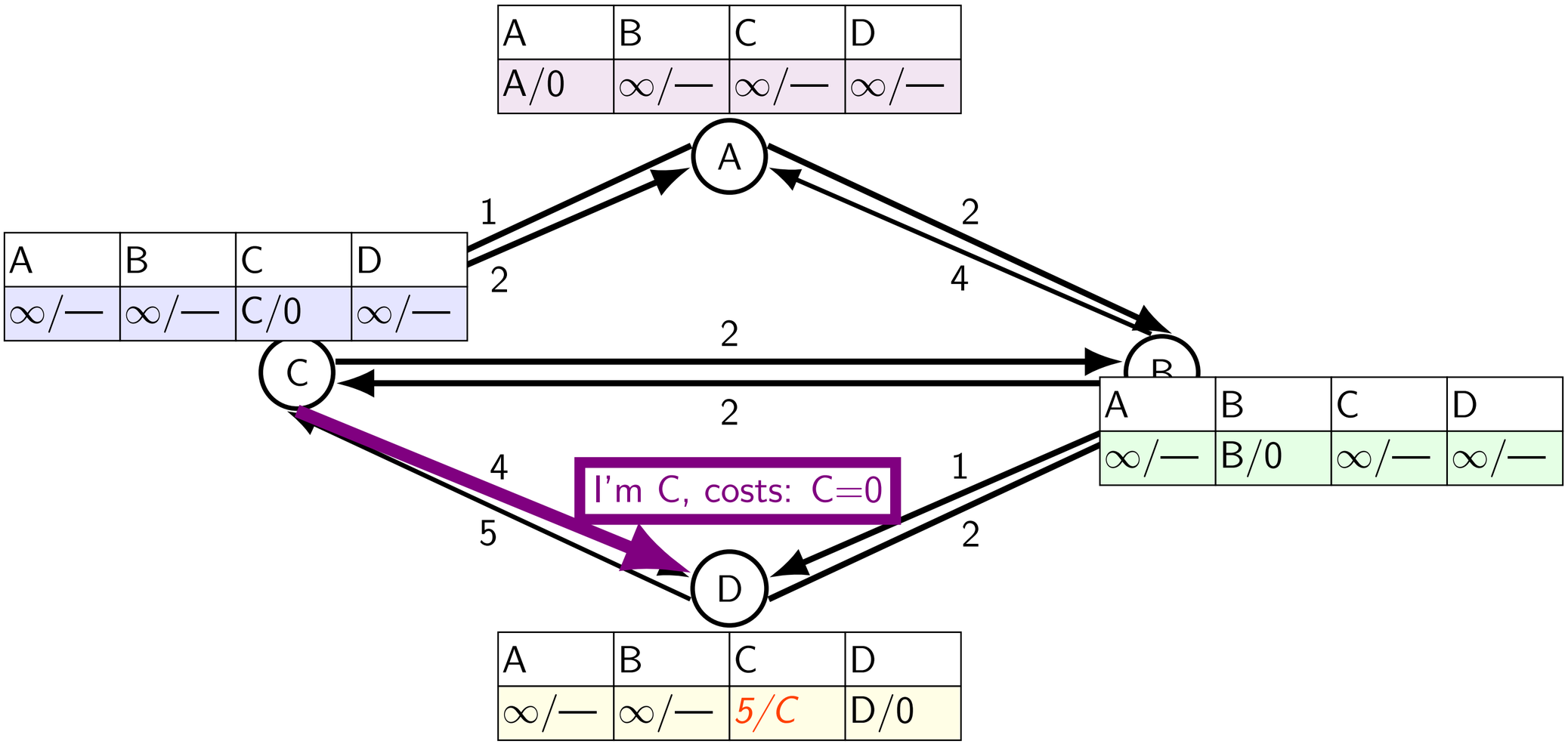


store table row = “distance vector” on each node

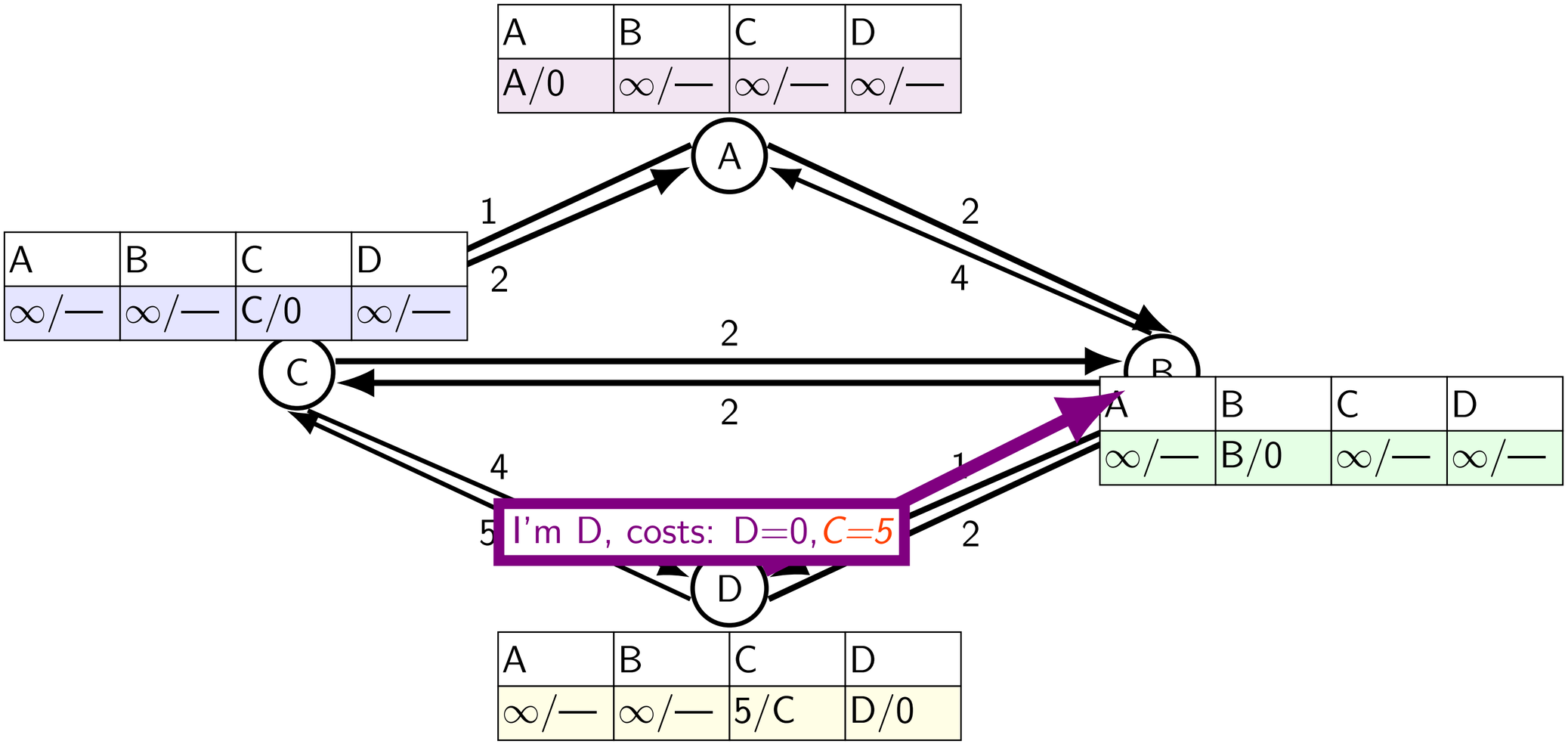
distributing Bellman-Ford



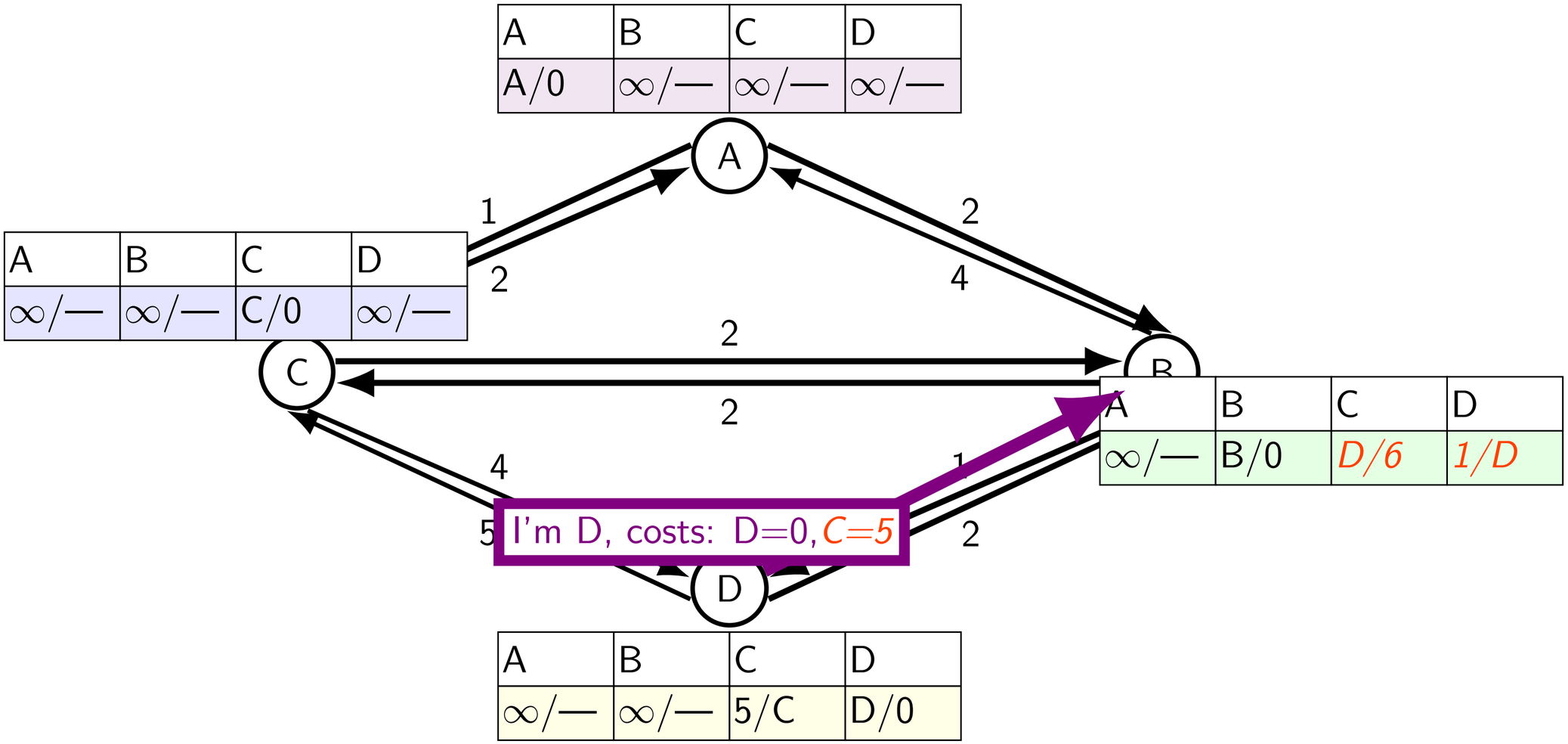
distributing Bellman-Ford



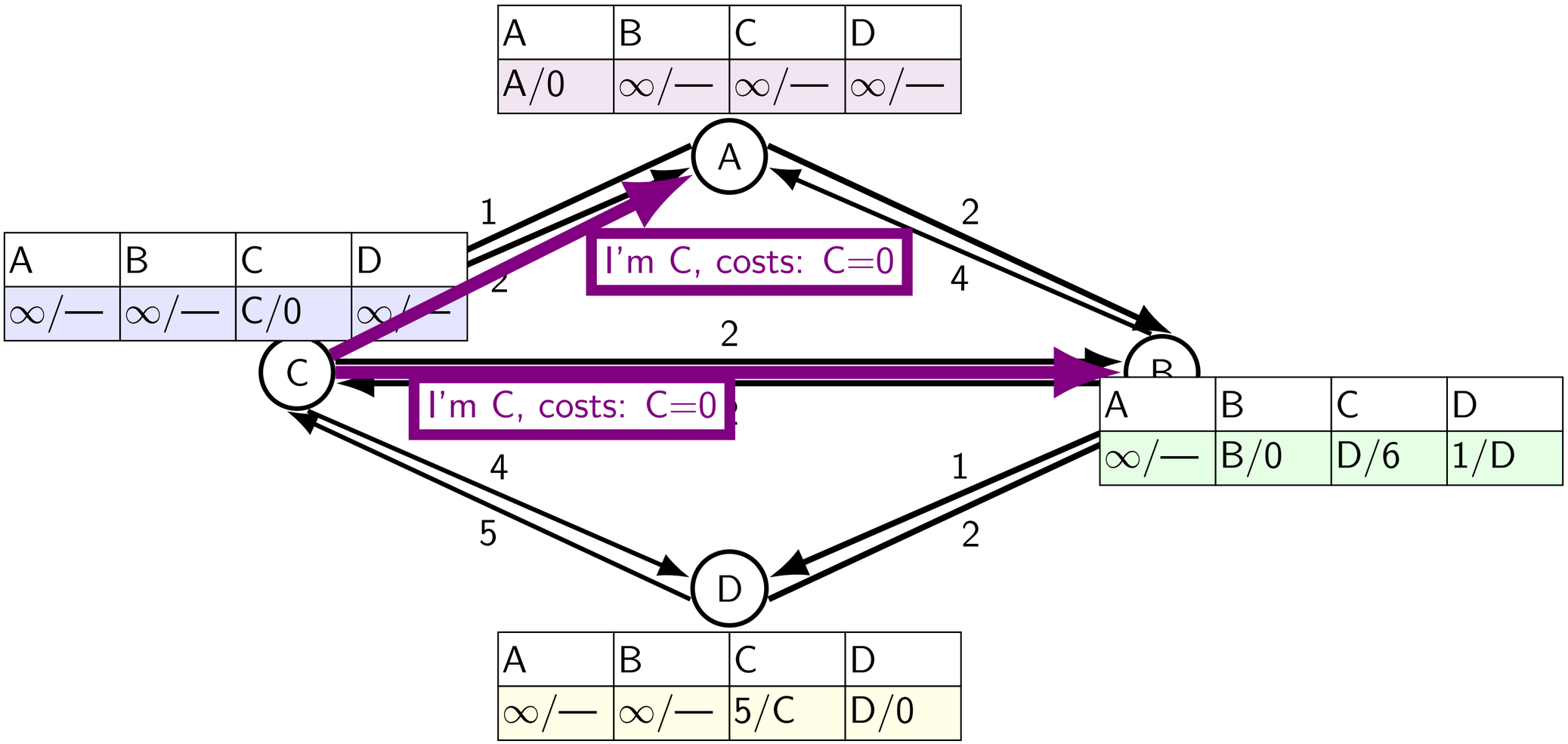
distributing Bellman-Ford



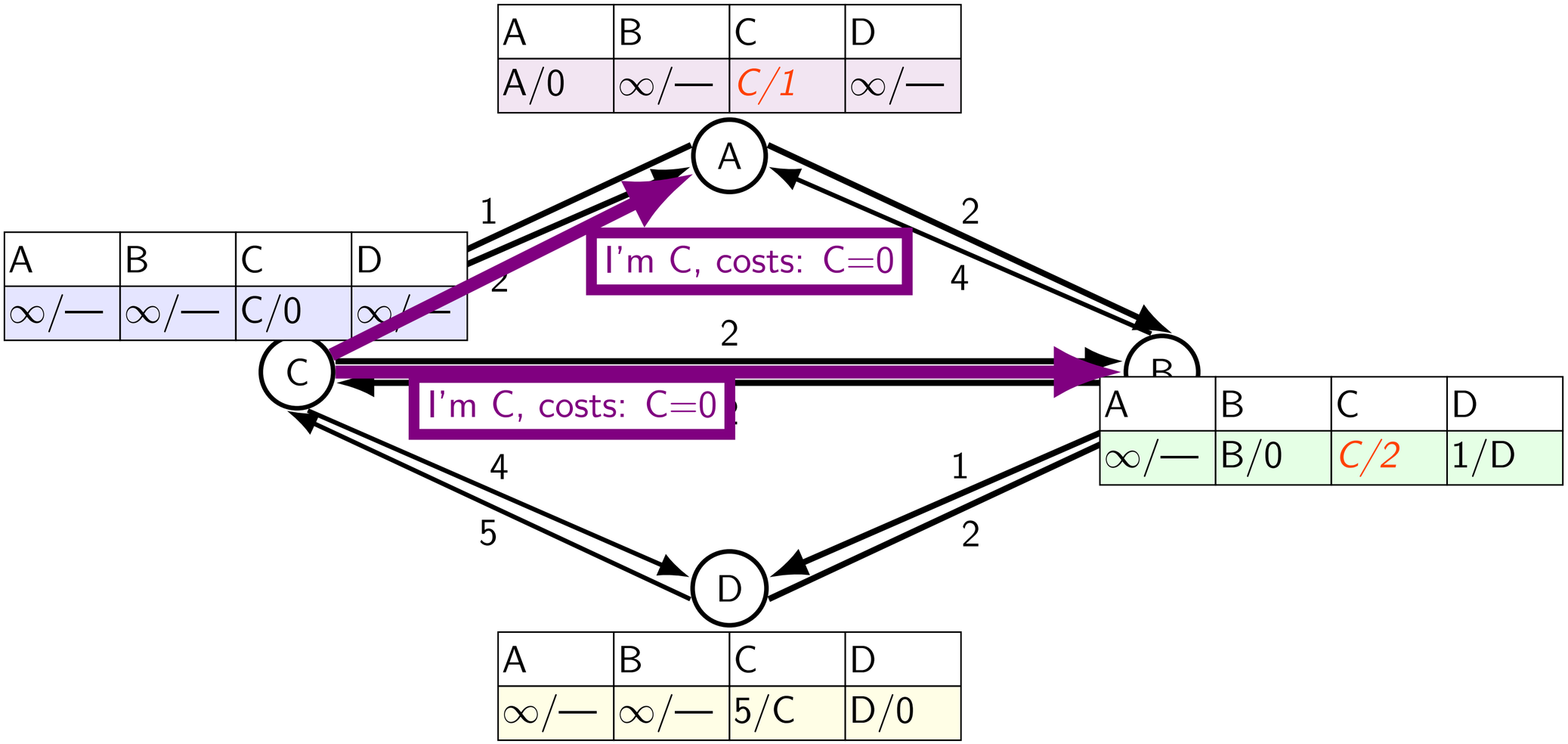
distributing Bellman-Ford



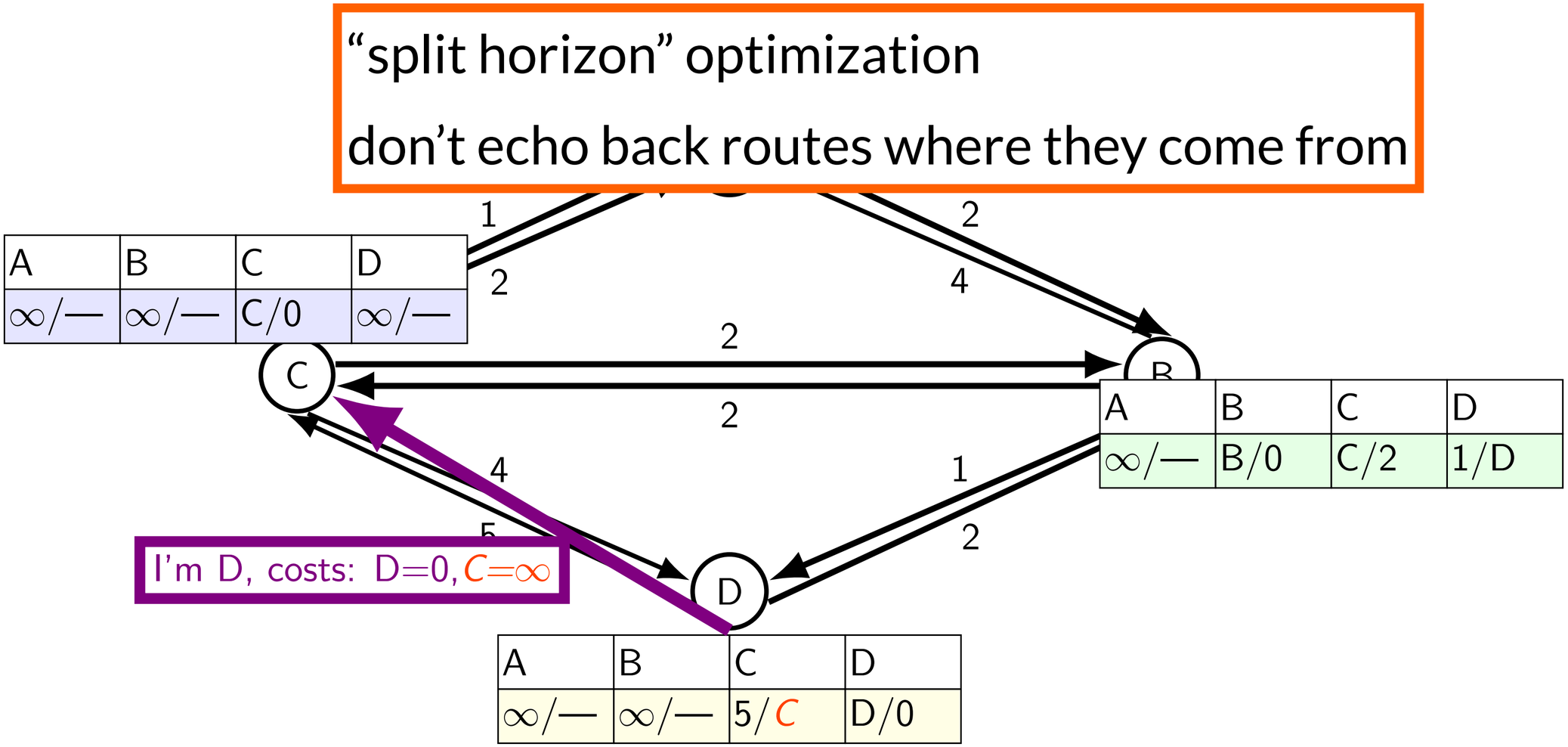
distributing Bellman-Ford



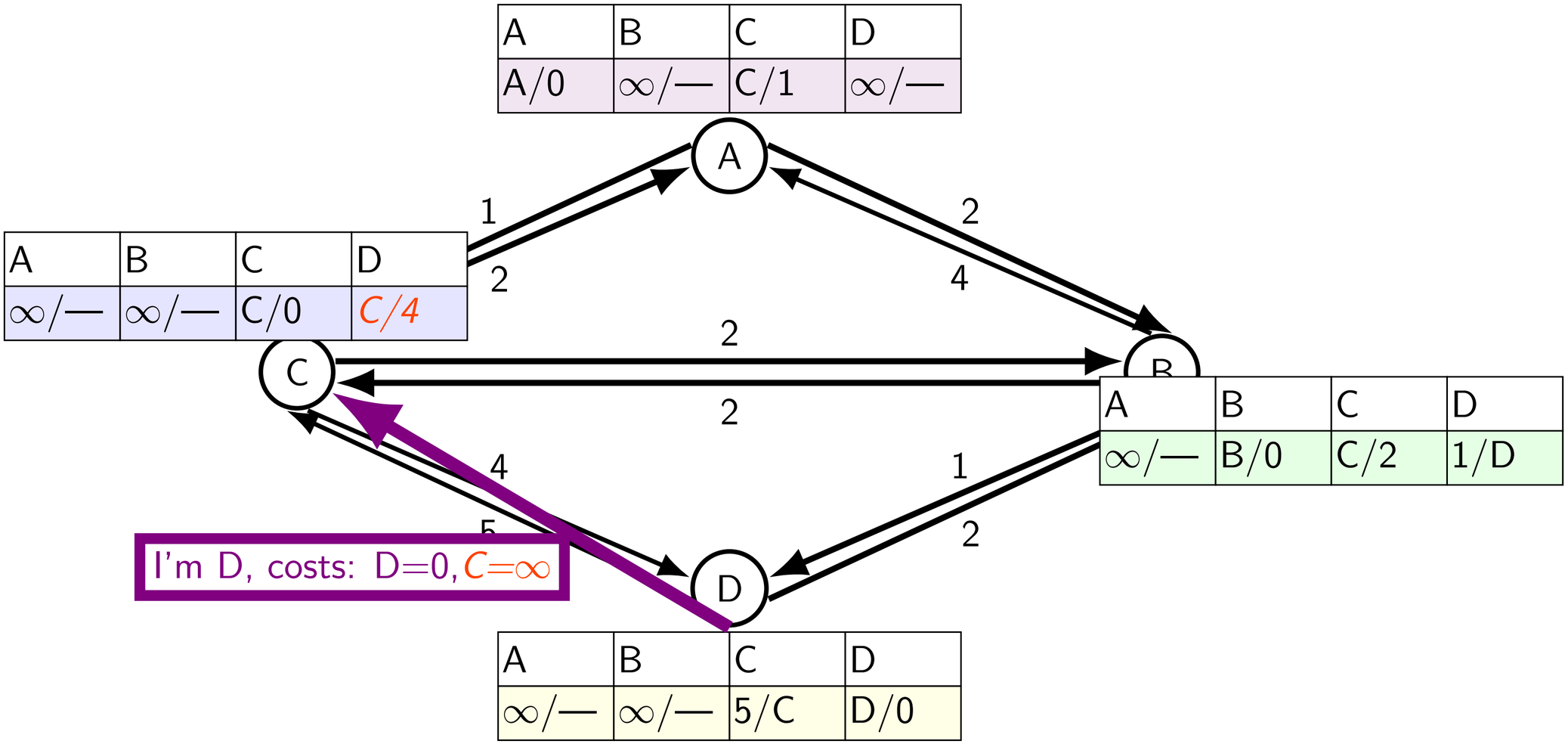
distributing Bellman-Ford



distributing Bellman-Ford



distributing Bellman-Ford



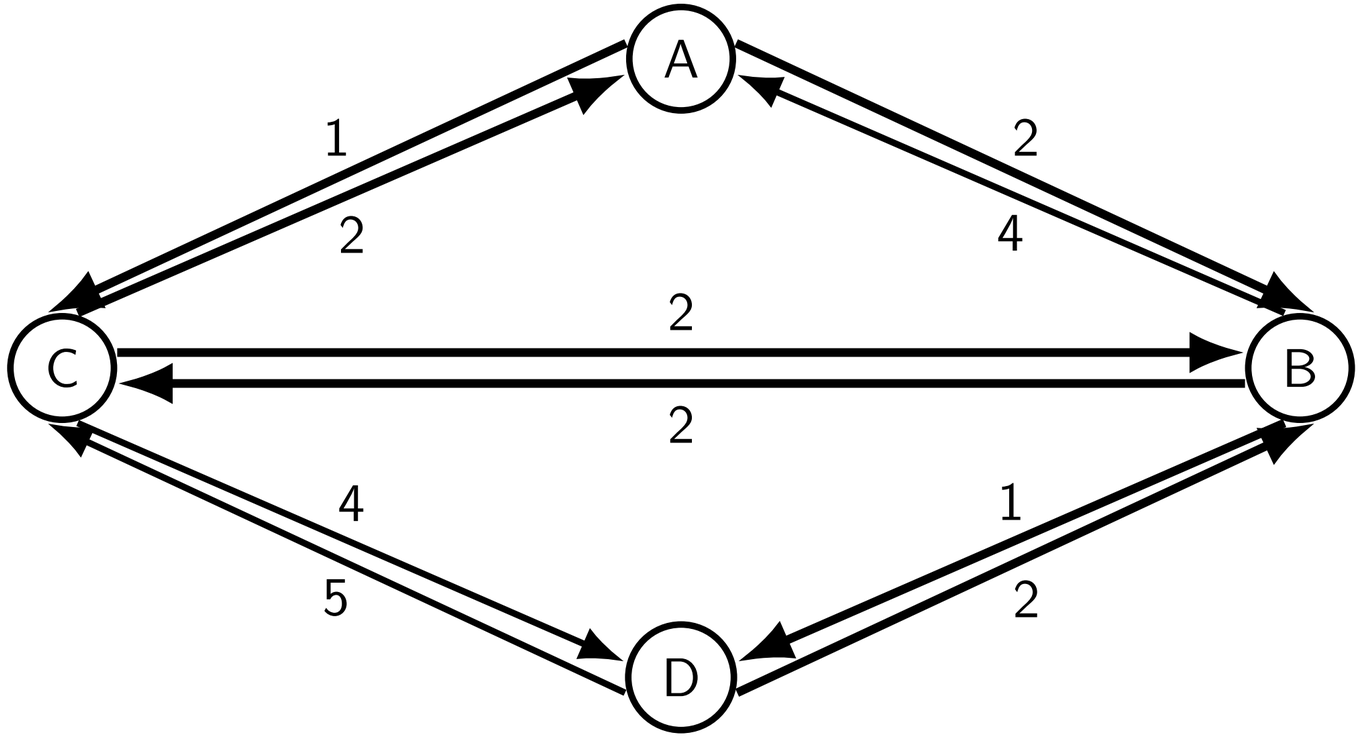
A	B	C	D
A/0	∞/—	C/1	∞/—

A	B	C	D
∞/—	∞/—	C/0	*C/4*

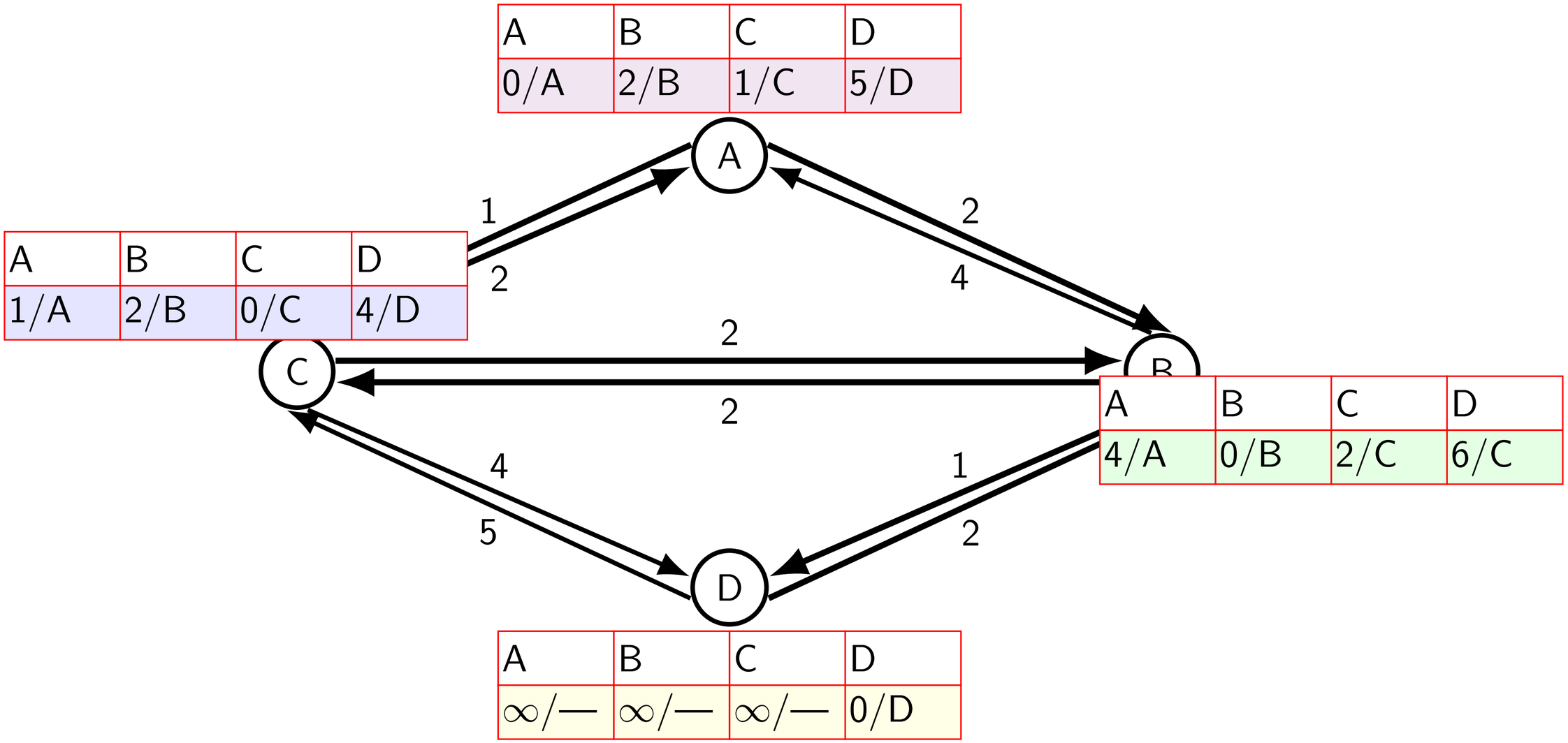
A	B	C	D
∞/—	B/0	C/2	1/D

A	B	C	D
∞/—	∞/—	5/C	D/0

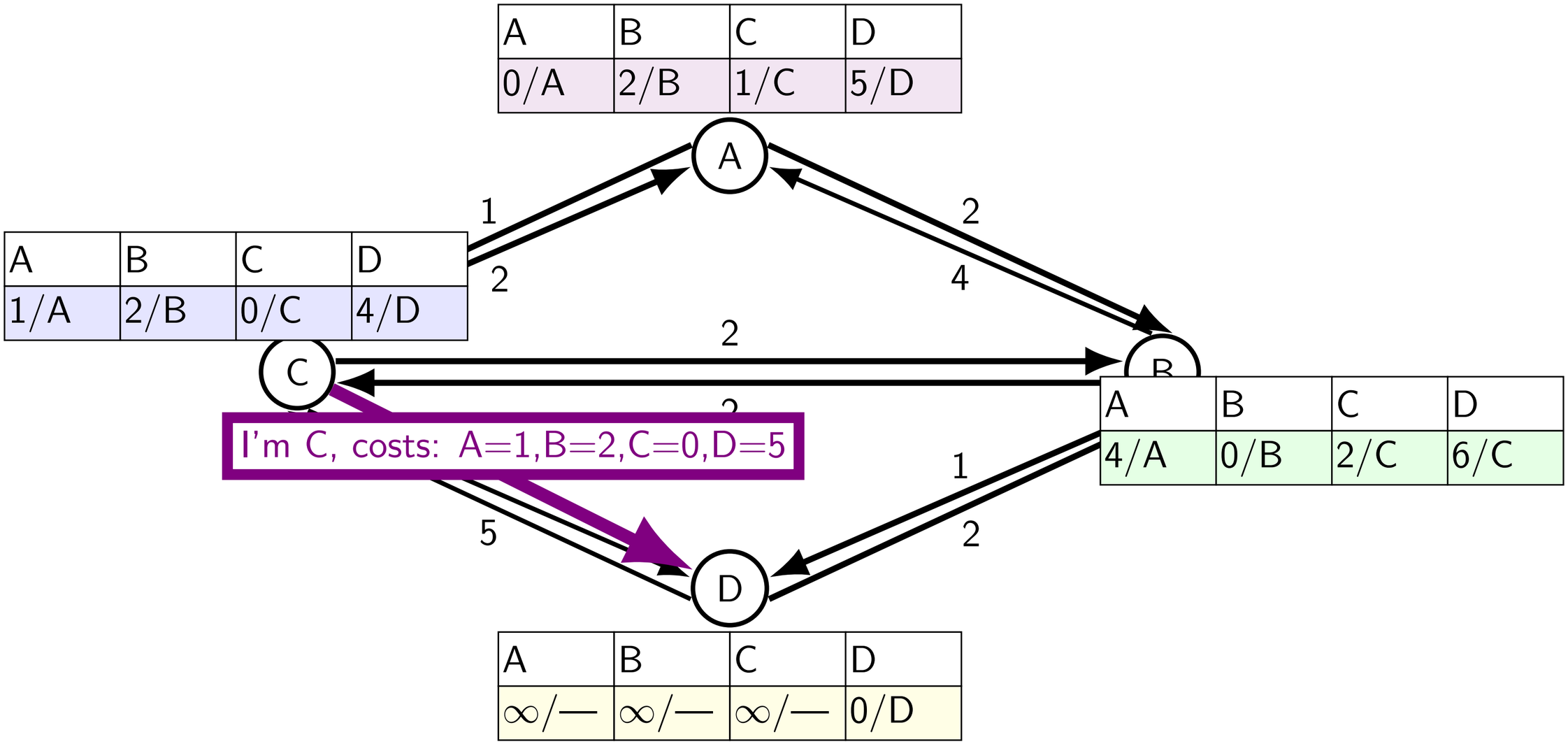
distributing Bellman-Ford (2)



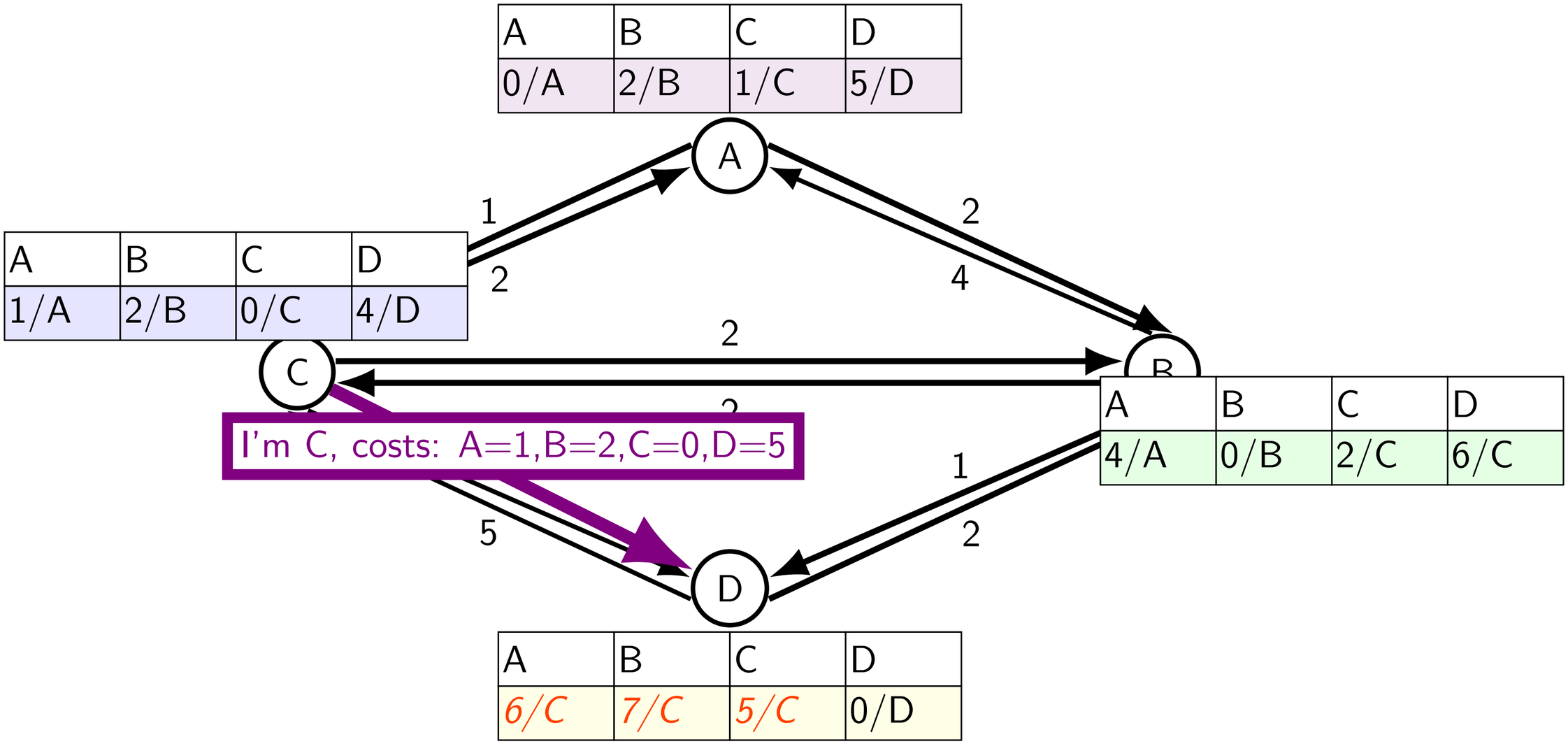
distributing Bellman-Ford (2)



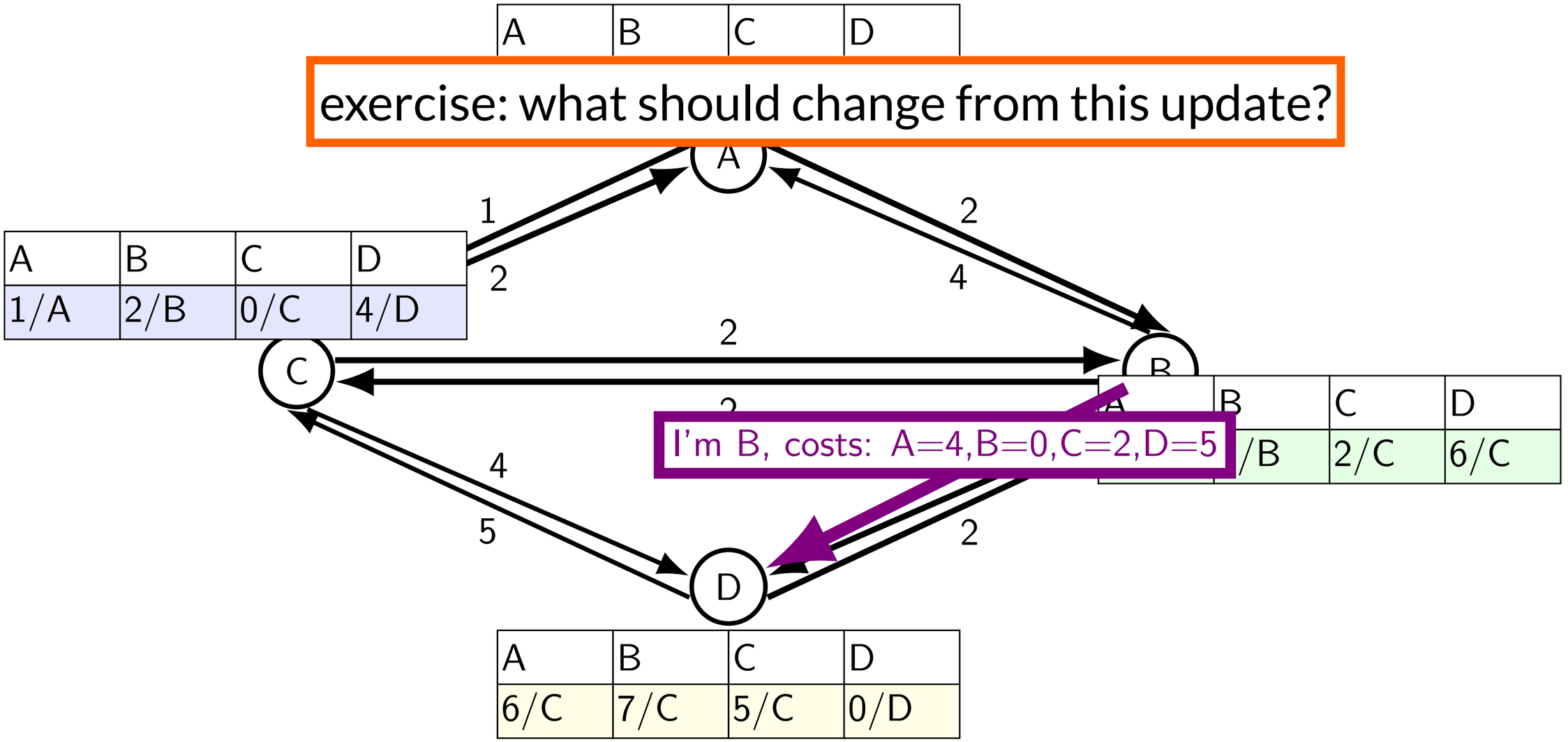
distributing Bellman-Ford (2)



distributing Bellman-Ford (2)



distributing Bellman-Ford (2)



networks v routers (DV)

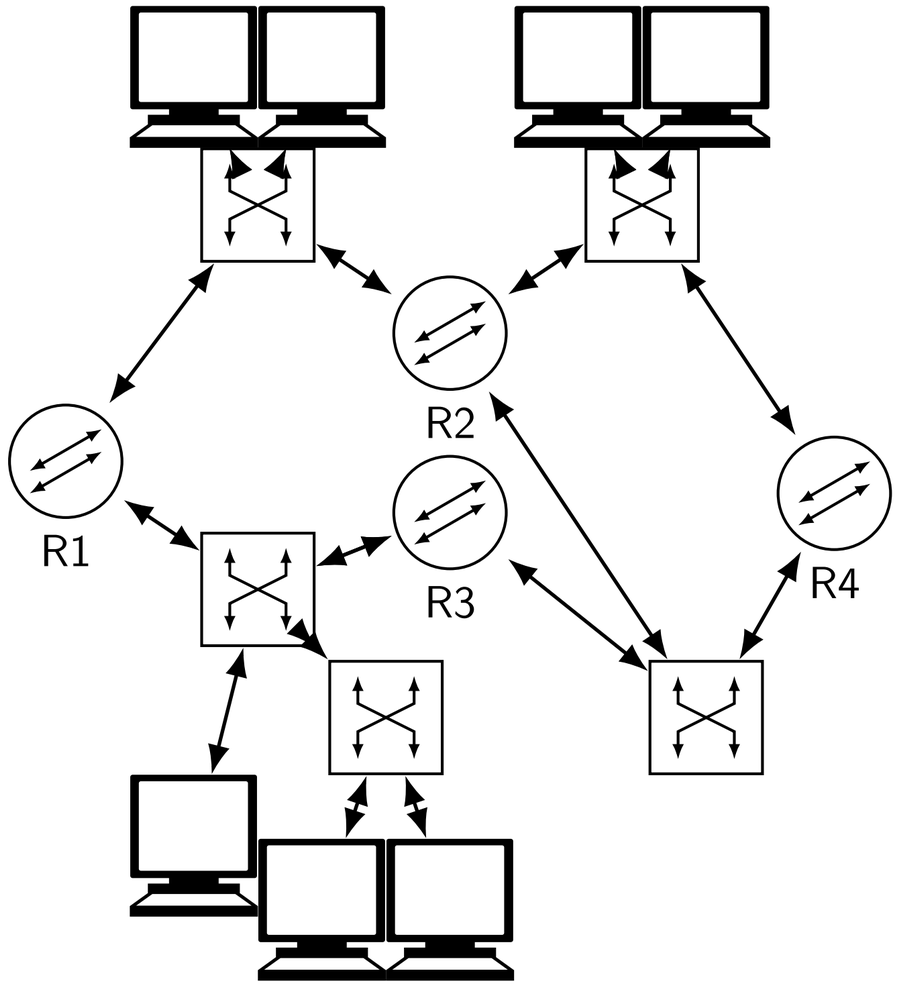
acting as if we want distance to routers

but really want distance to networks

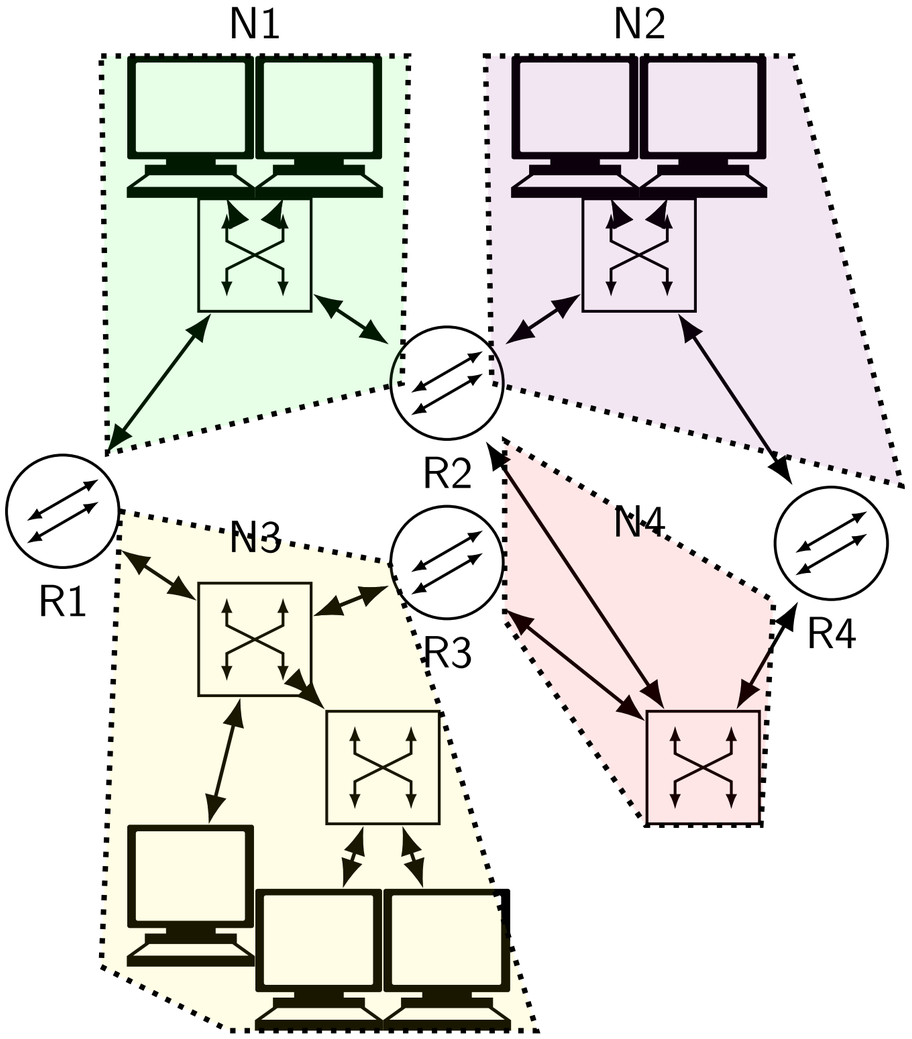
distance vectors will track distance to *networks*

but next hops will be routers

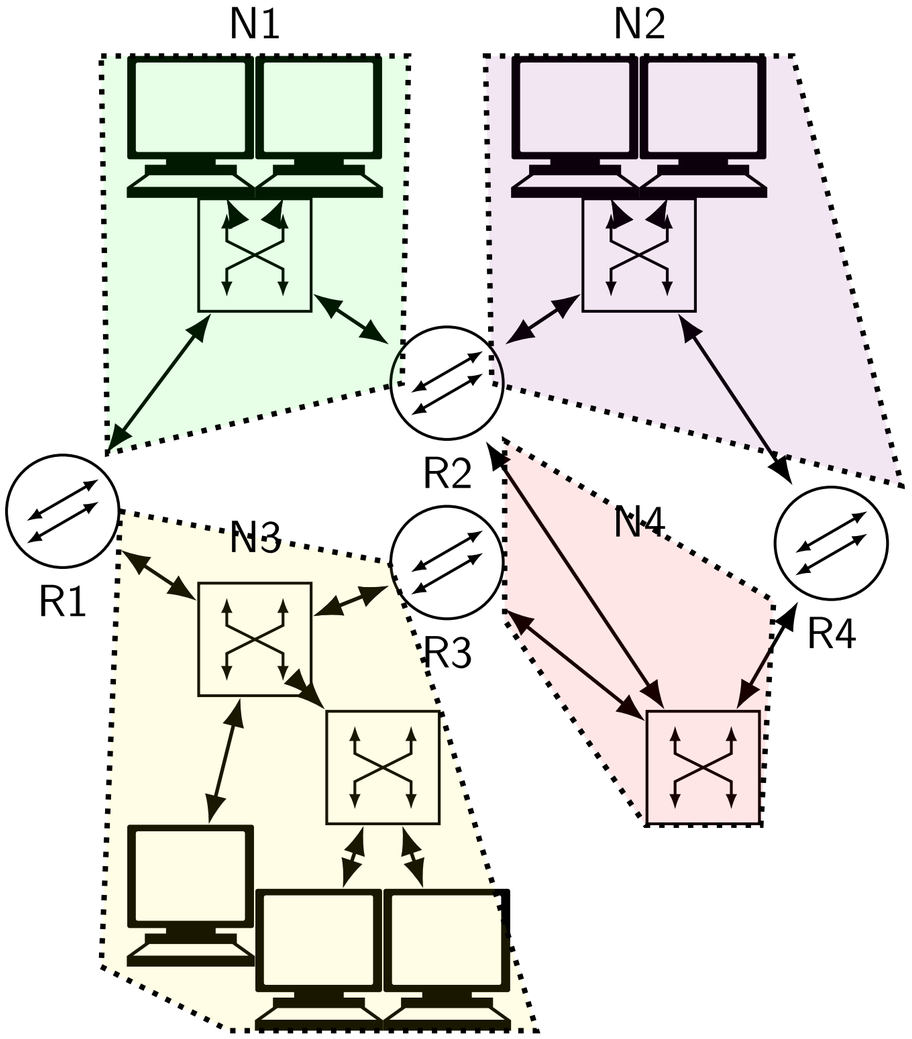
networks v routers (DV)

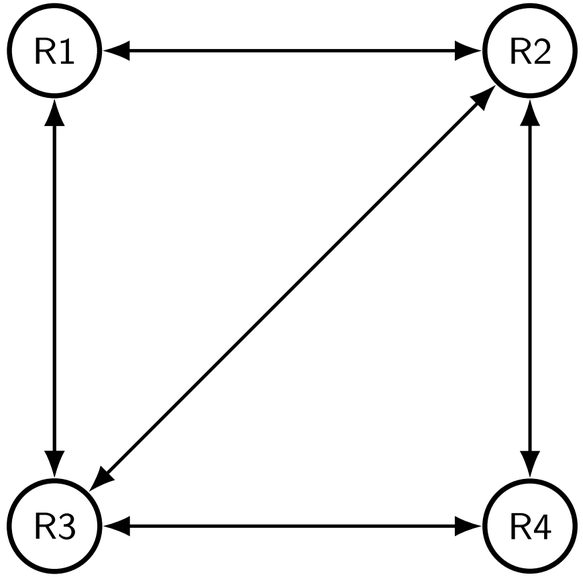


networks v routers (DV)

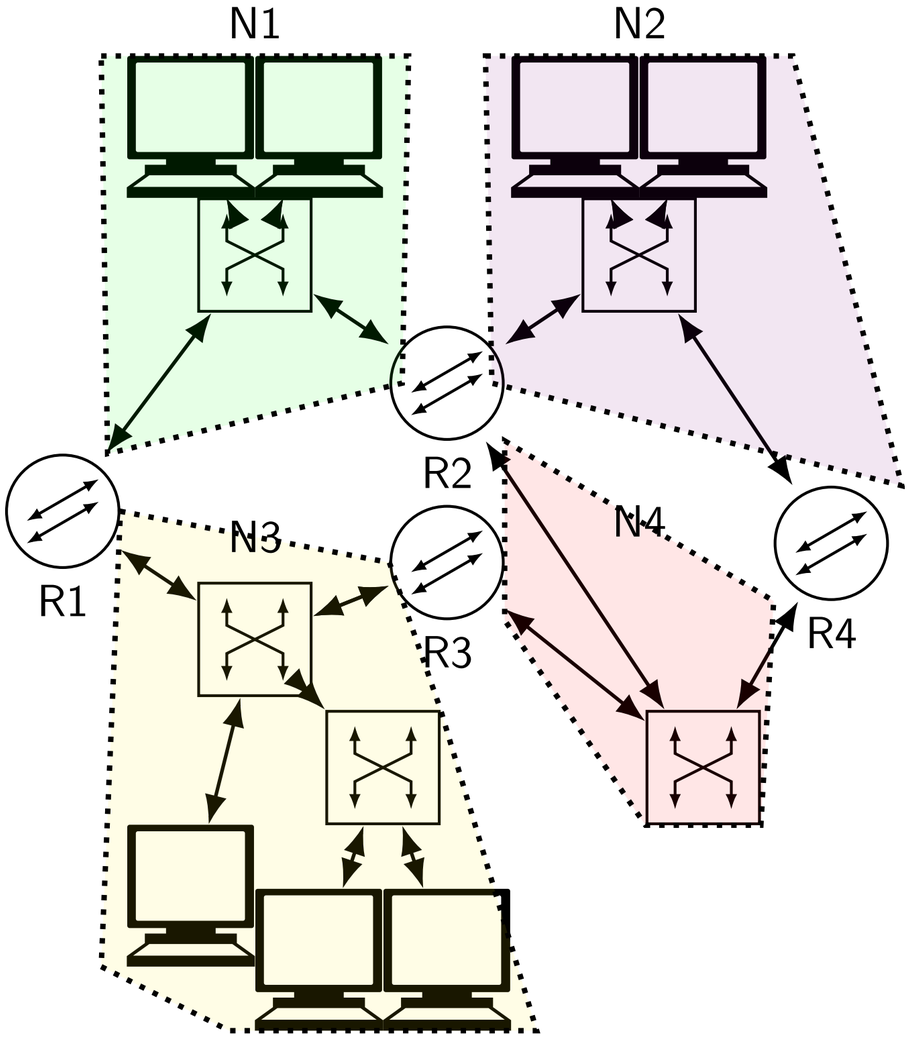


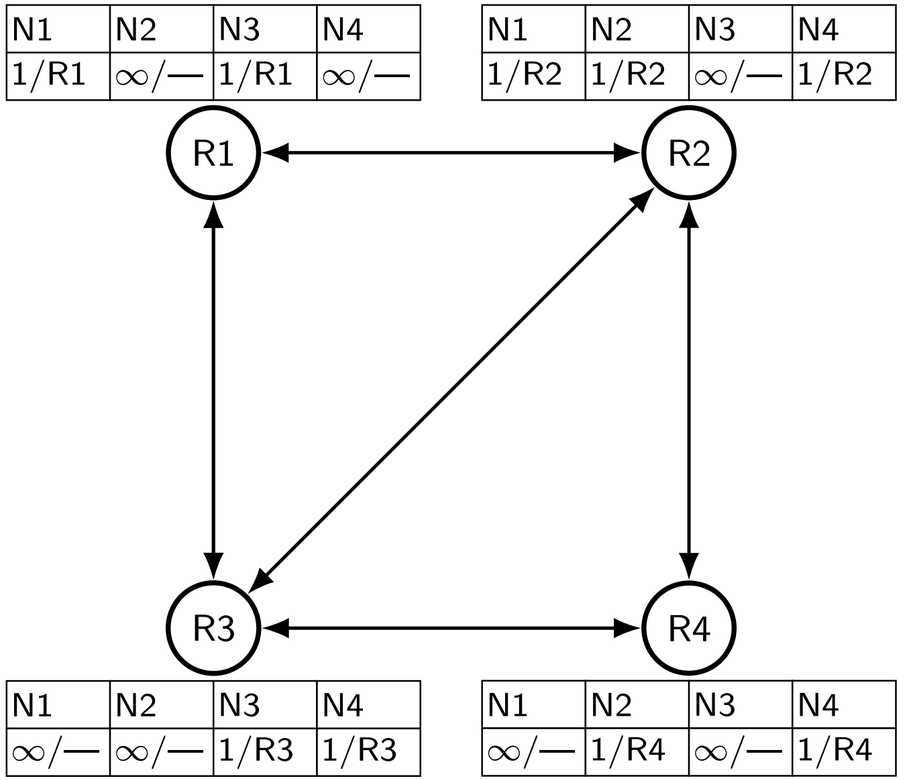
networks v routers (DV)



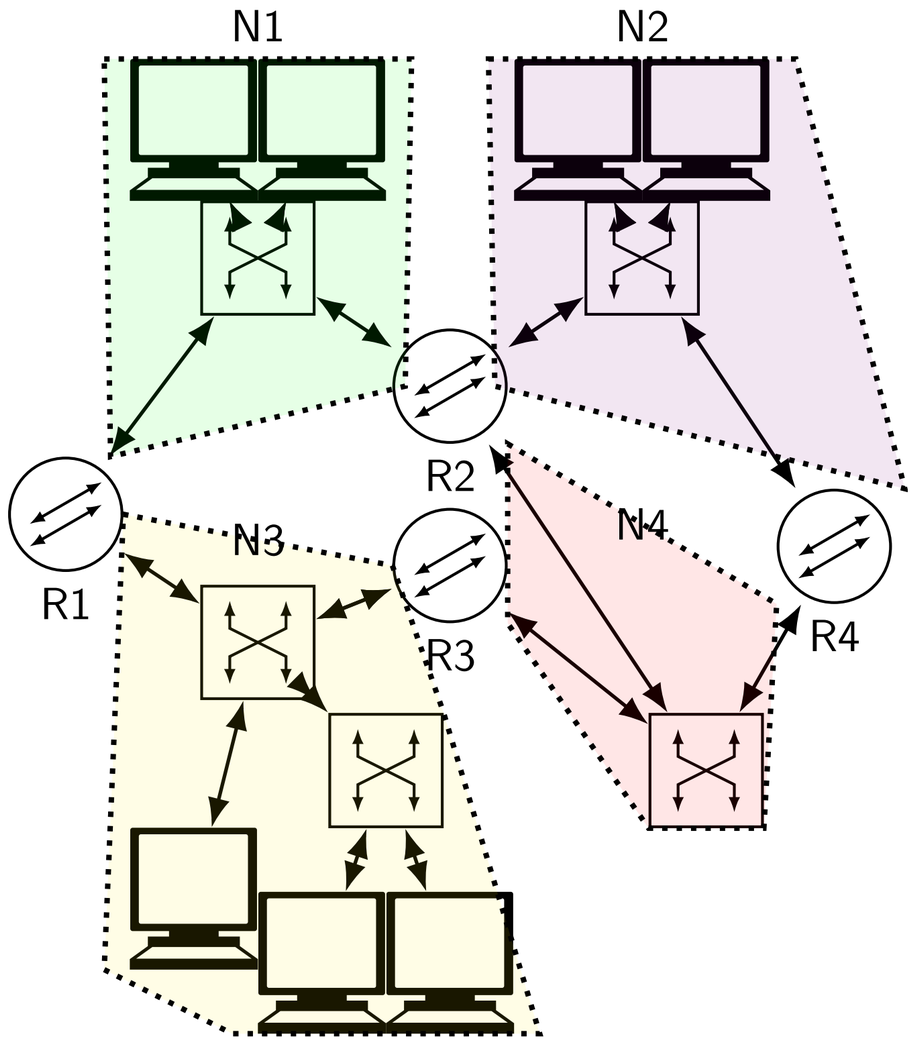


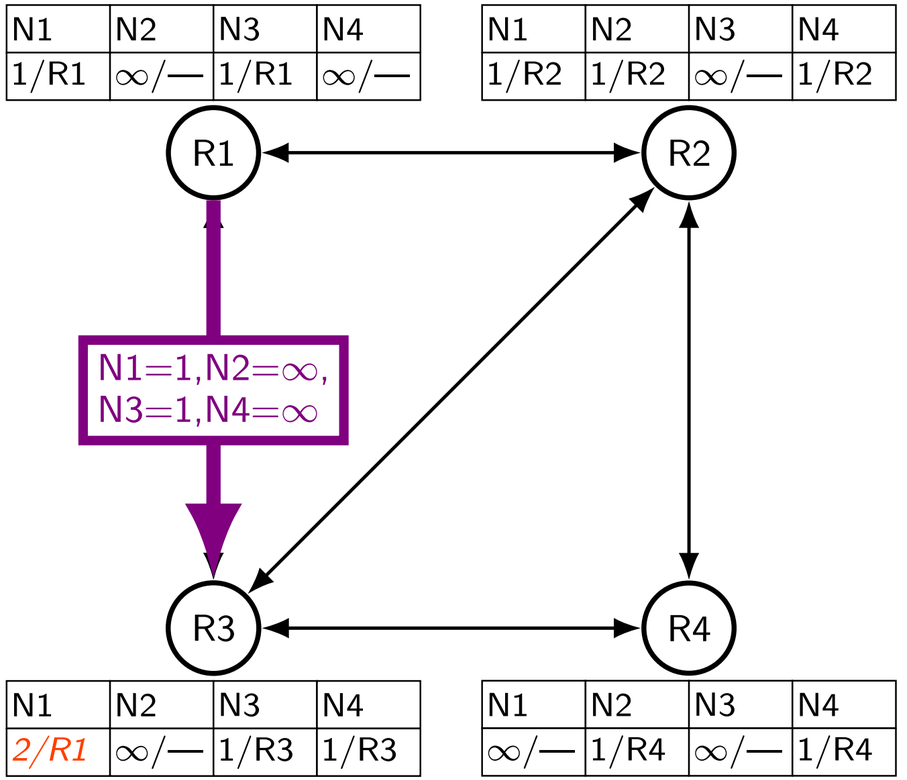
networks v routers (DV)



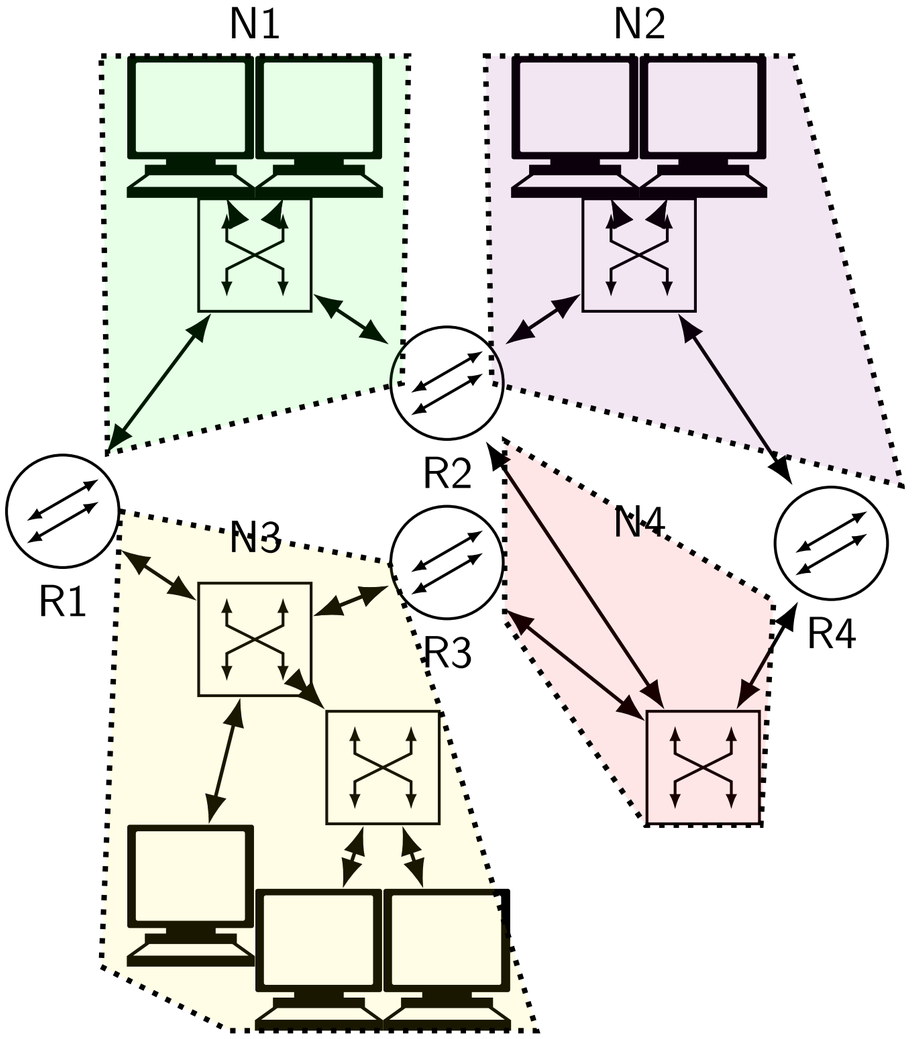


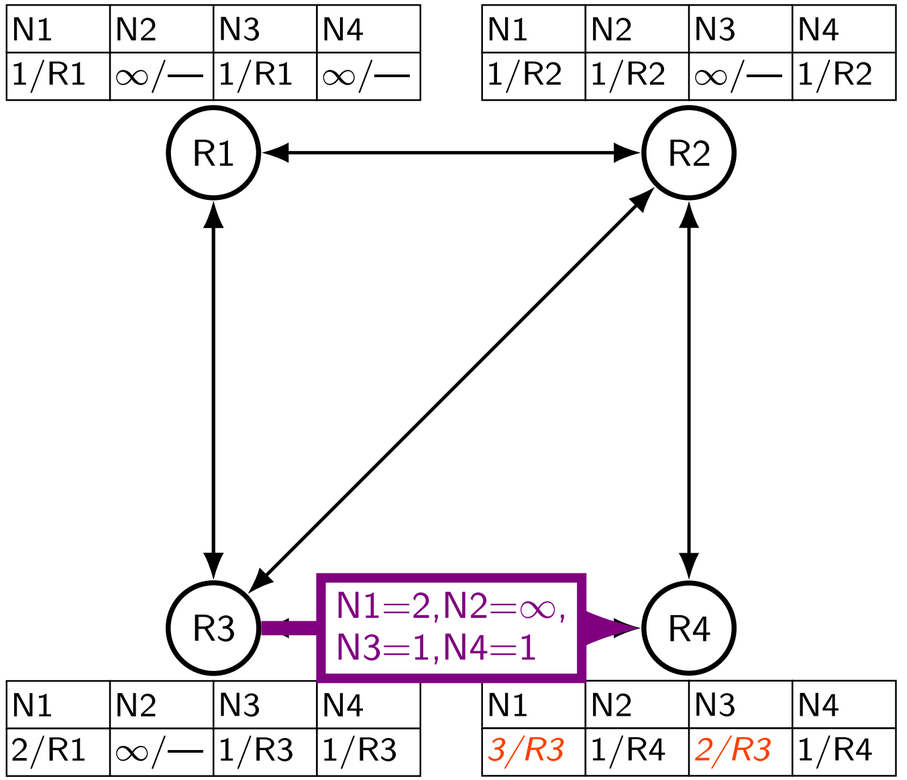
networks v routers (DV)





networks v routers (DV)





distance vector routing

each node keeps *distance vector*

- distance to each other node (network)
- also which neighbor to go through to get that distance

periodically send distance vector to all neighbors

when receiving distance vector from X, check

"would going through X give me a better distance?"

- if so, update distance + which neighbor

Routing Information Protocol

router broadcast on networks it's connected to packet containing list of:

- networks it can reach (example: 1.2.3.0/24)
- its next hop to that network
- its metric (distance) to reach that network

each router on that network processes that packet

on receiving distances, routers see if they can update their routes

- routes will be to networks (1.2.3.0/24, etc.), not routers

local information

routers need to track themselves:

which networks they can reach directly

(which networks is it connected to)

the ‘distance’ it needs to reach those networks

(probably based on its bandwidth to that network?)

RIP — when to update

policy: every approx. 30 seconds always AND

immediately on changes (“triggered”)

means that connecting new router should better routes quickly

links going down

problem with our update rule:

assumes routes only get better

reality: sometimes links go down

need to find different route

updating for removal (1)

let's say I'm A and my distance vector is:

A=0 via A, B=4 via B, C=5 via D, D=4 via D

if my link to D goes down, new distance vector should be?

A=0 via A, B=4 via B, C=∞ via no one, D=∞ via no one

later updates might fix ∞s

updating for removal (2)

let’s say I’m A and my distance vector is:

B=4 via B, C=5 via D, D=4 via D

and D tells me its distance vector is

B=8 via A, C=∞ via no one, D=0 via D

then my (A)’s new distance vector should be?

B=4 via B, C=∞ via no one, D=4 via D

updating for removal (3)

let’s say I’m A and my distance vector is:

B=4 via B, C=5 via D, D=4 via D

and D tells me its distance vector is

B=8 via A, C=5 via A, D=0 via D

then my (A)’s new distance vector should be?

B=4 via B, C=∞ via no one, D=4 via D

updating for removal (4)

let’s say I’m A and my distance vector is:

B=4 via B, C=5 via D, D=4 via D

and D tells me its distance vector is

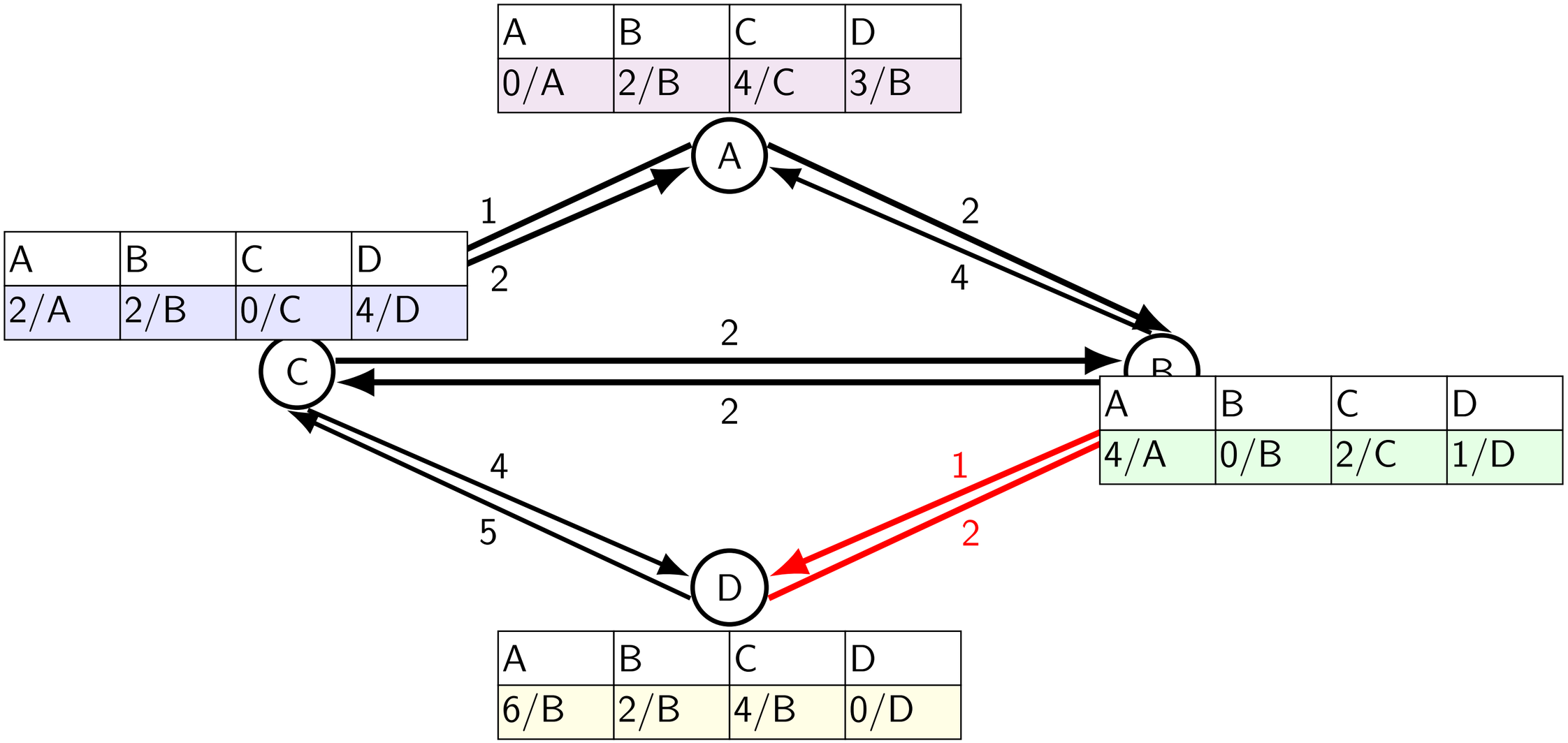
B=3 via B, C=8 via B, D=0 via D

then my (A)’s new distance vector should be?

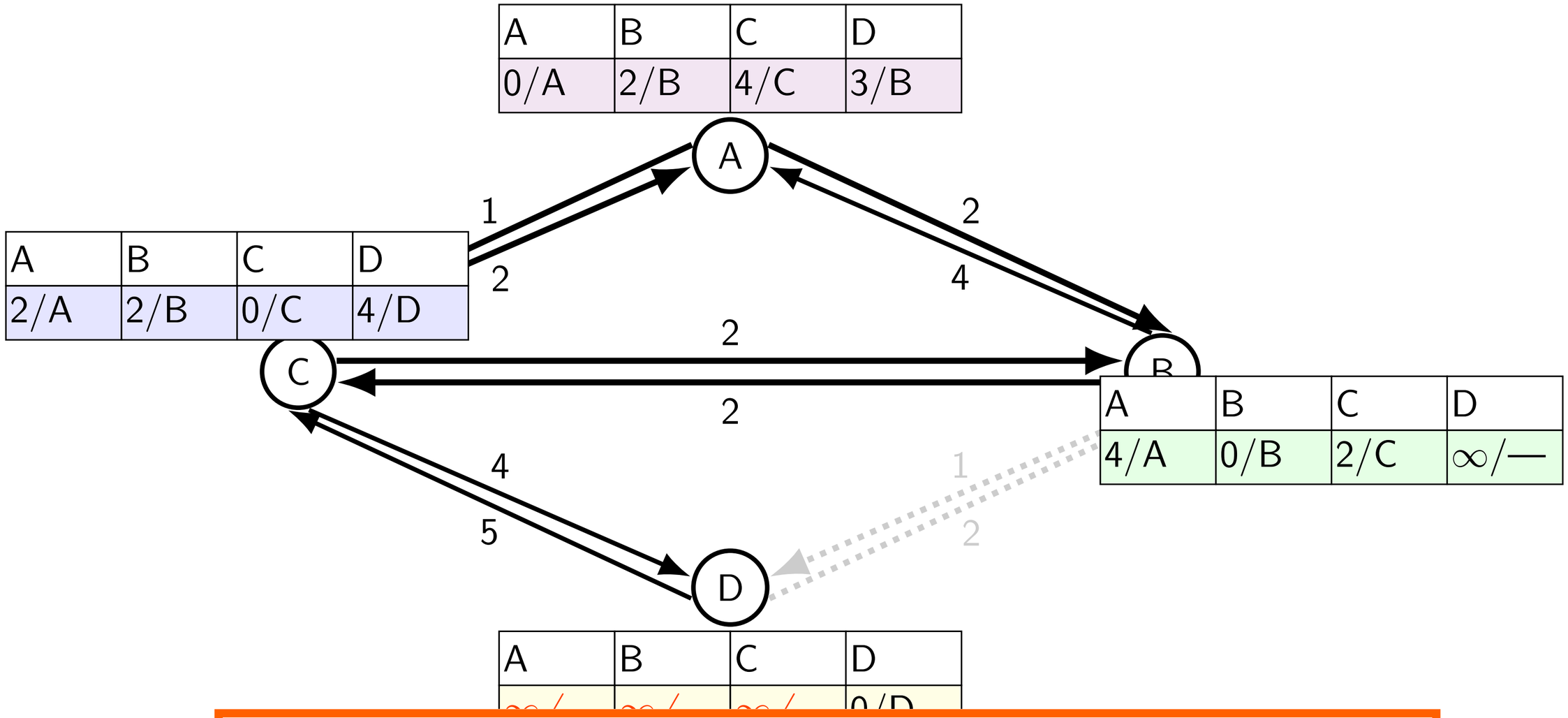
B=4 via B, C=12 via D, D=4 via D

probably later update from B will overwrite route to C

removal?



removal?



will need additional update messages to find new routes

split horizon with poison reverse

when sending distance vectors, ‘posion’ routes to same node

- make sure other node won’t go back to us…
- only to have us go back to them

example, if I’m A and routes are:

- A: 0 via A; B: 2 via B; C: 4 via C; D: 6 via B

when sending to B send:

- A: 0; B: 2; C: 4; D: ∞

without split horizon?

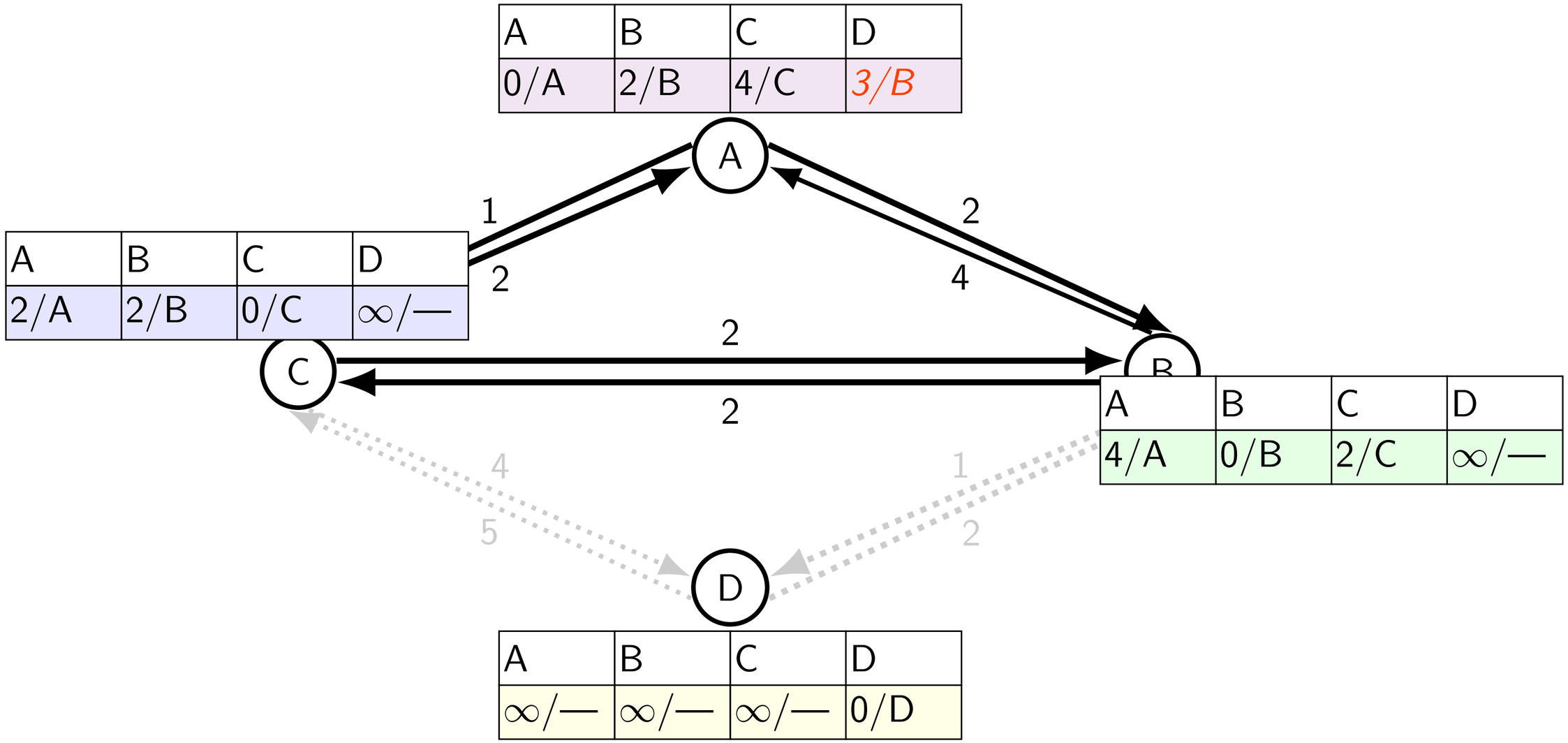
can create routing loop

example: if D unreachable from B, then B goes to A and A goes to B

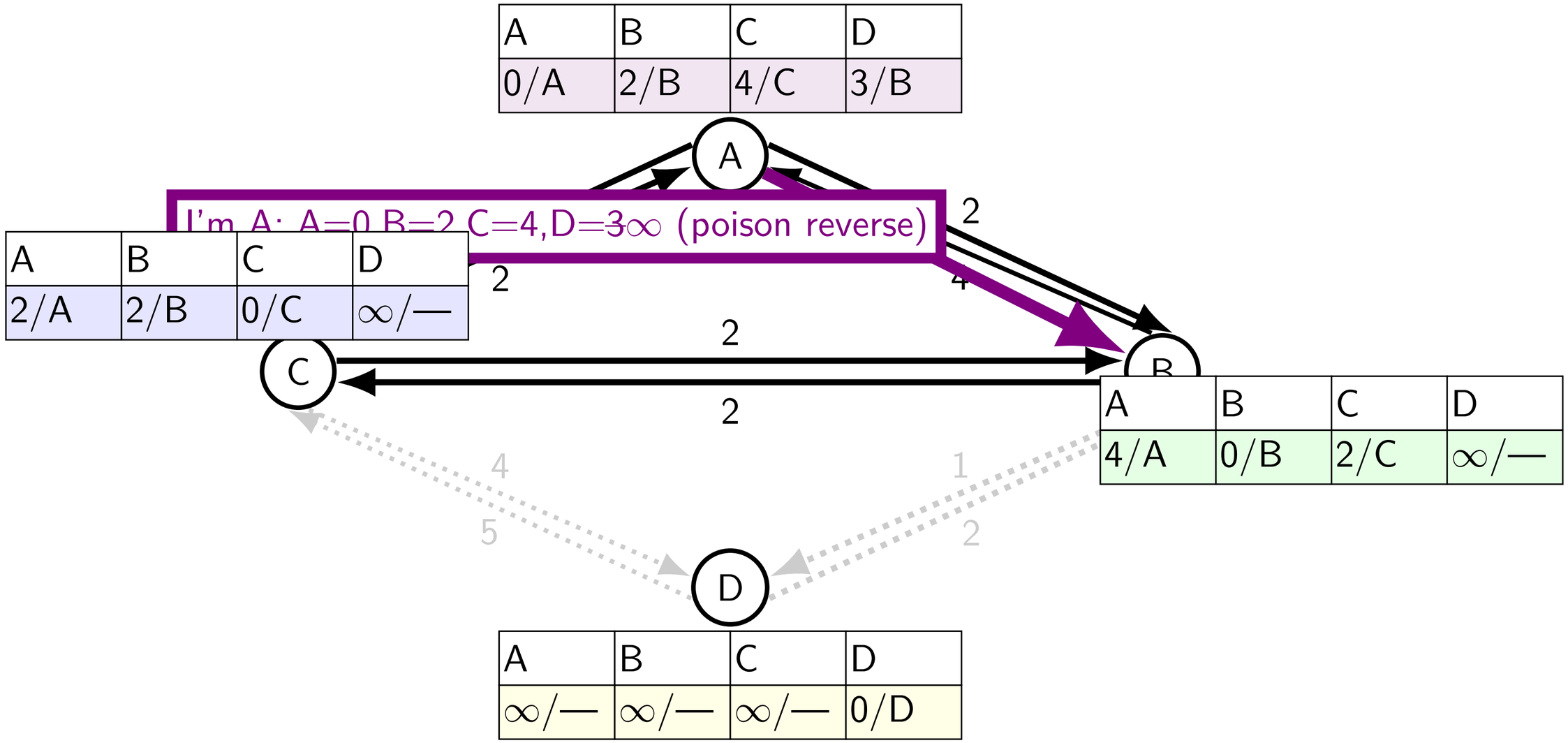
called “count-to-infinity” problem

because A, B will keep updating distance higher and higher

avoided count-to-infinity



avoided count-to-infinity



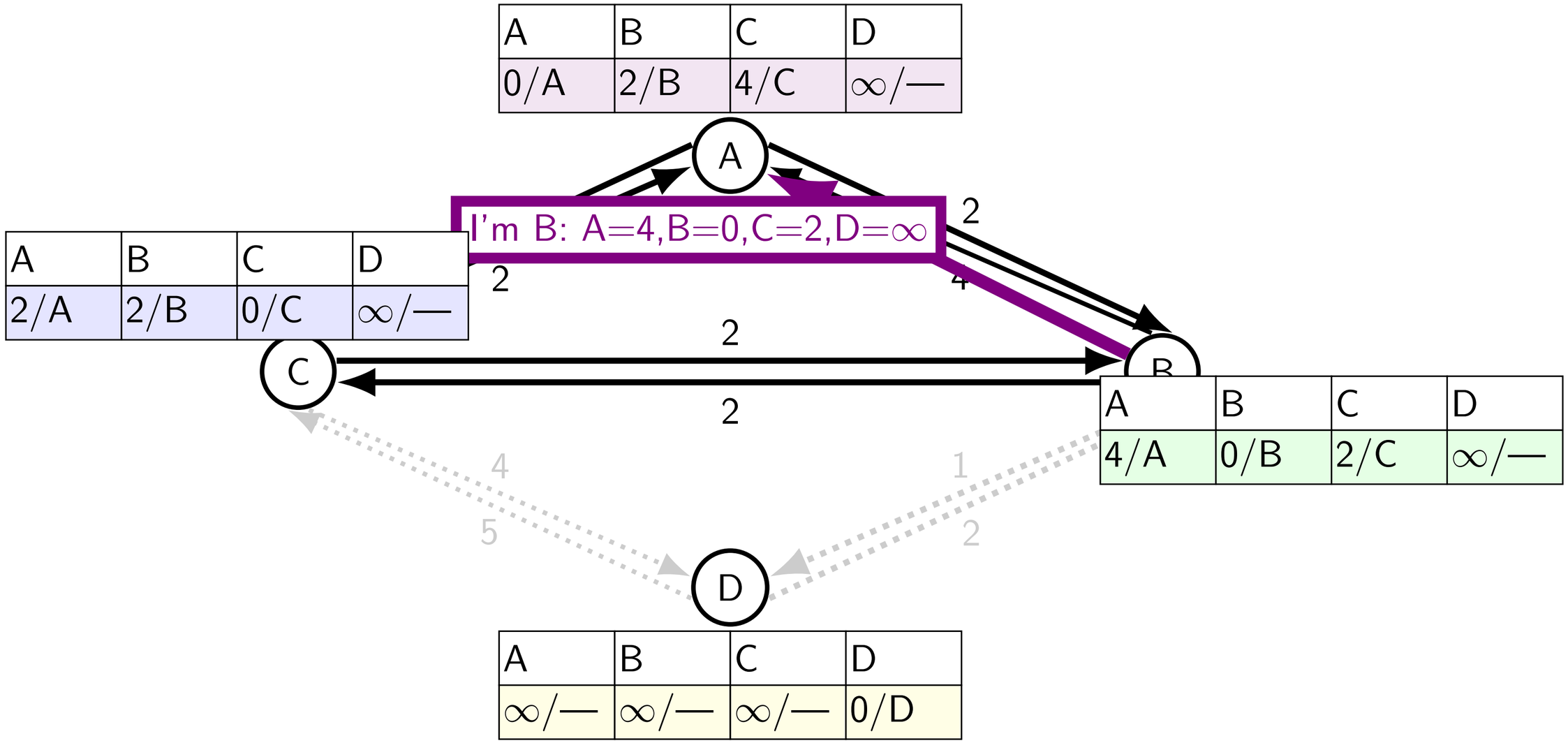
A	B	C	D
0/A	2/B	4/C	3/B

A	B	C	D
2/A	2/B	0/C	∞/—

A	B	C	D
4/A	0/B	2/C	∞/—

A	B	C	D
∞/—	∞/—	∞/—	0/D

avoided count-to-infinity



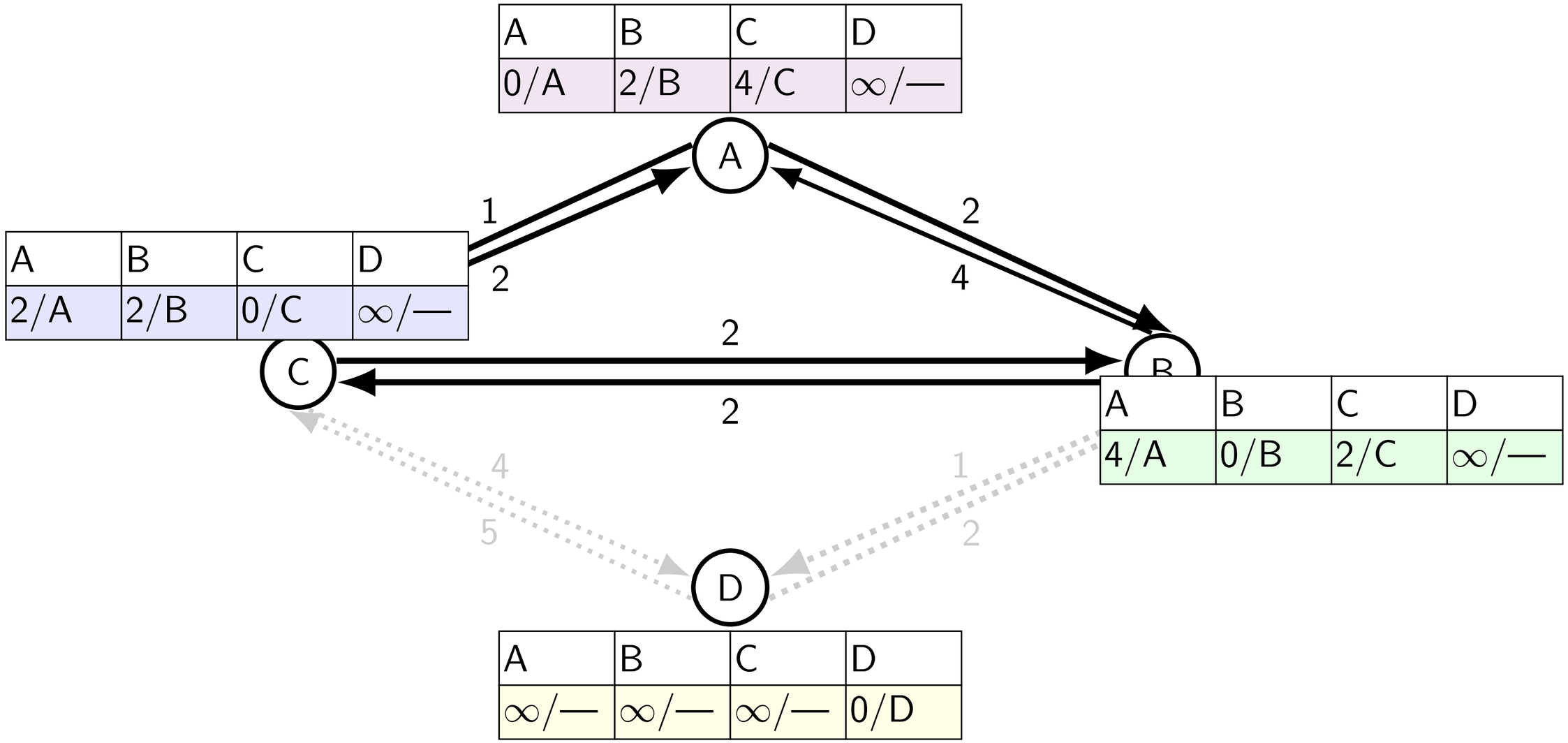
A	B	C	D
0/A	2/B	4/C	∞/—

A	B	C	D
2/A	2/B	0/C	∞/—

A	B	C	D
4/A	0/B	2/C	∞/—

A	B	C	D
∞/—	∞/—	∞/—	0/D

avoided count-to-infinity



trivial loop

oops: A to B to A to B to A to B to …

this case: relatively easy to avoid

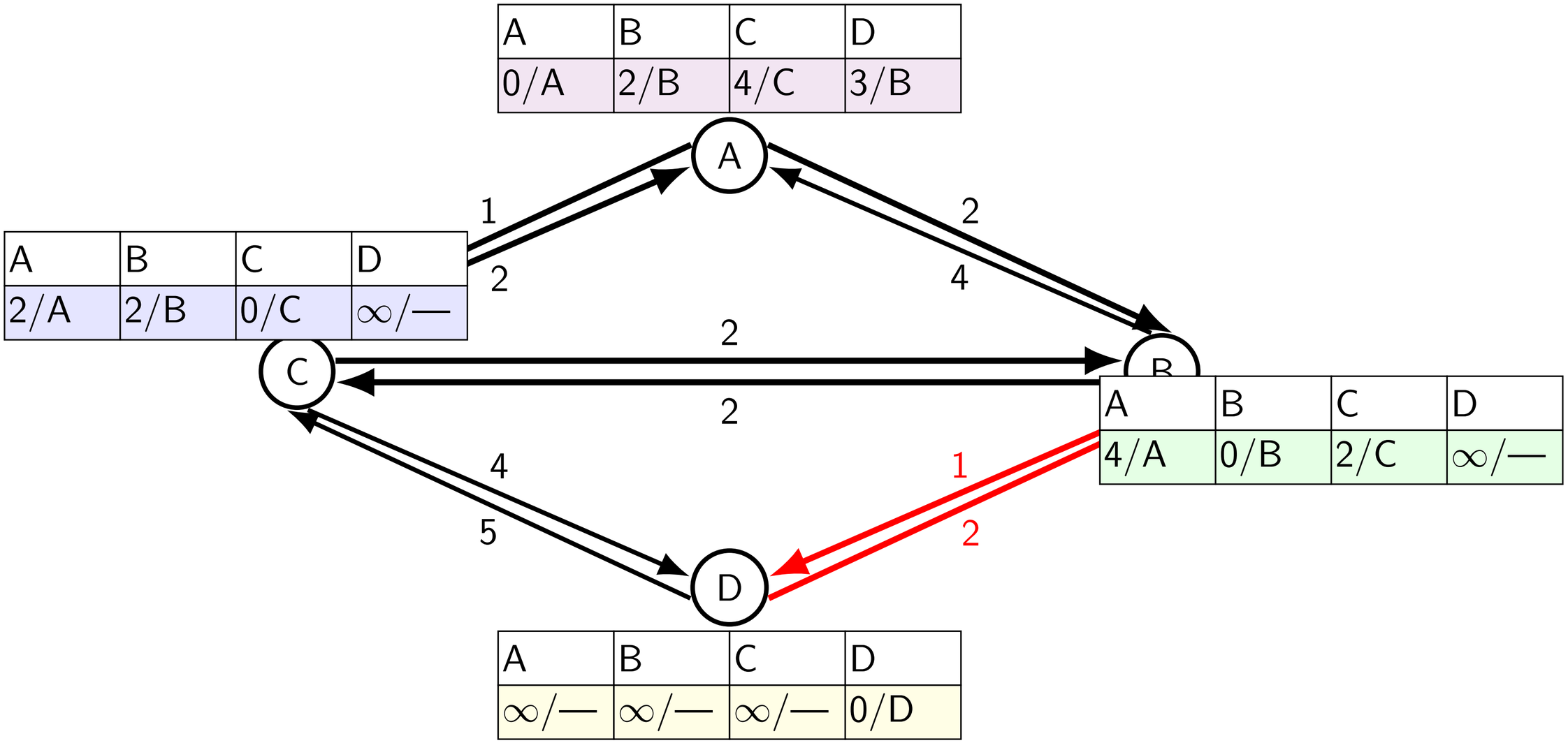
split horizon incomplete solution

split horizon prevents trivial loops, but…

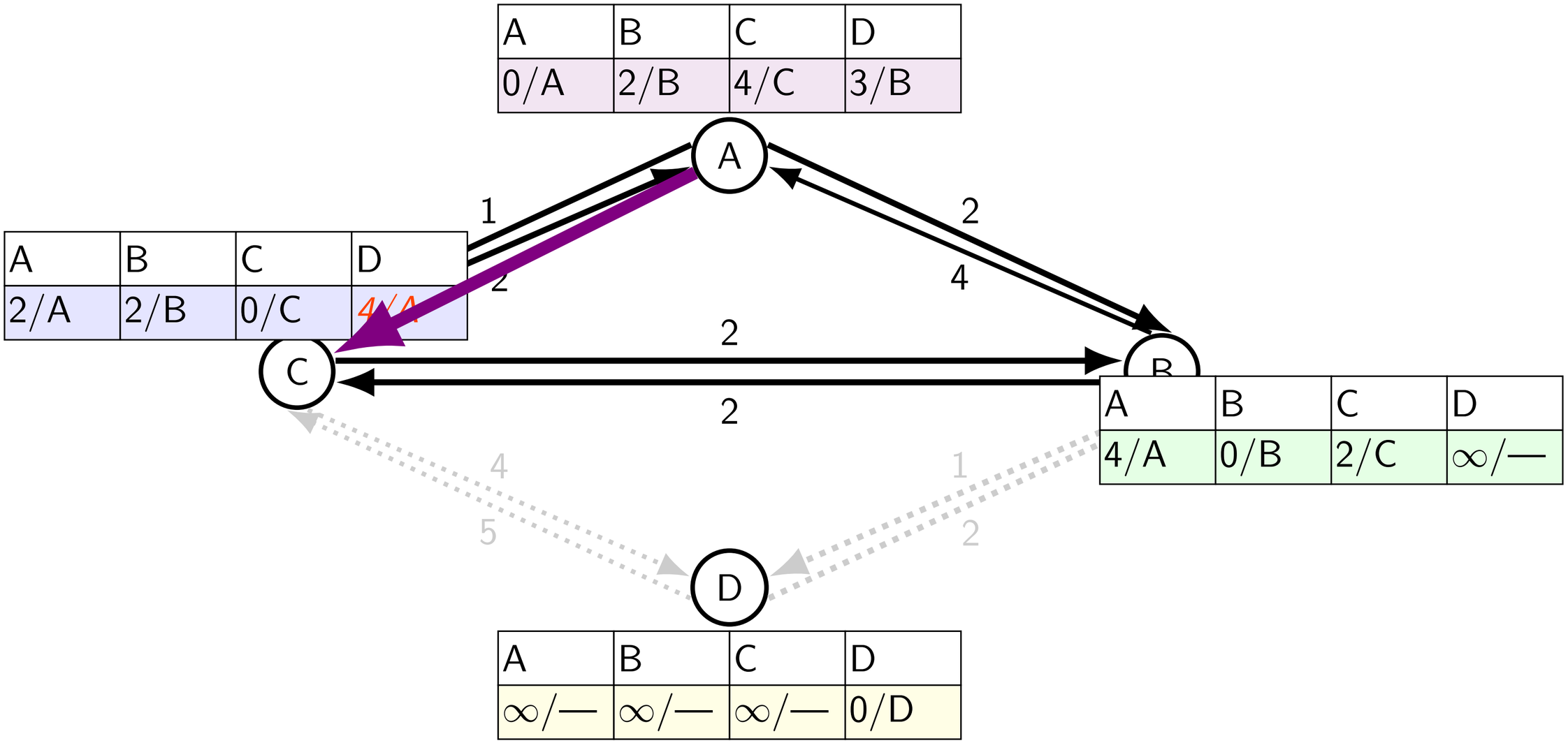
doesn’t actually solve the count-to-infinty problem

in well-connected network, there will be longer loops

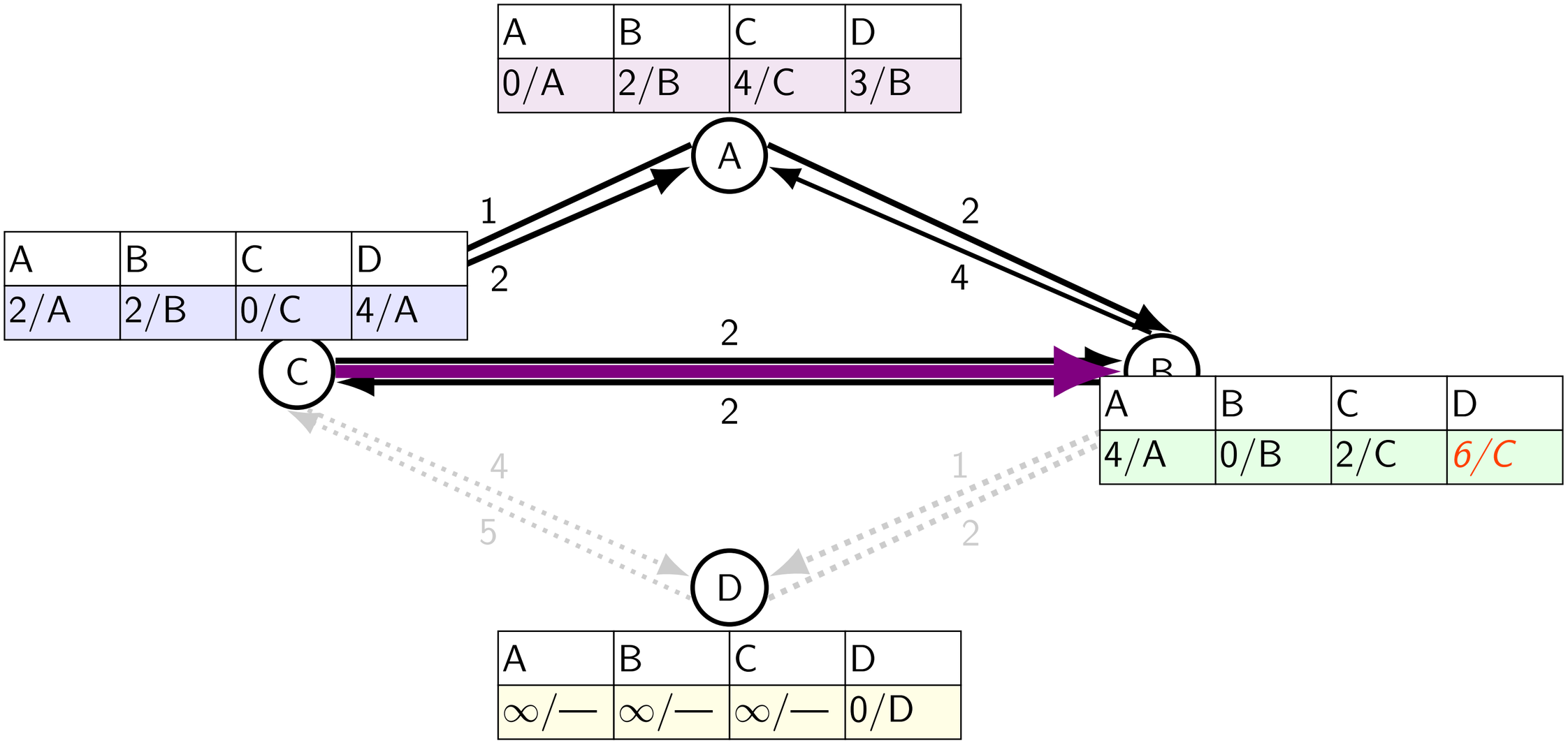
count-to-infinity (v2)



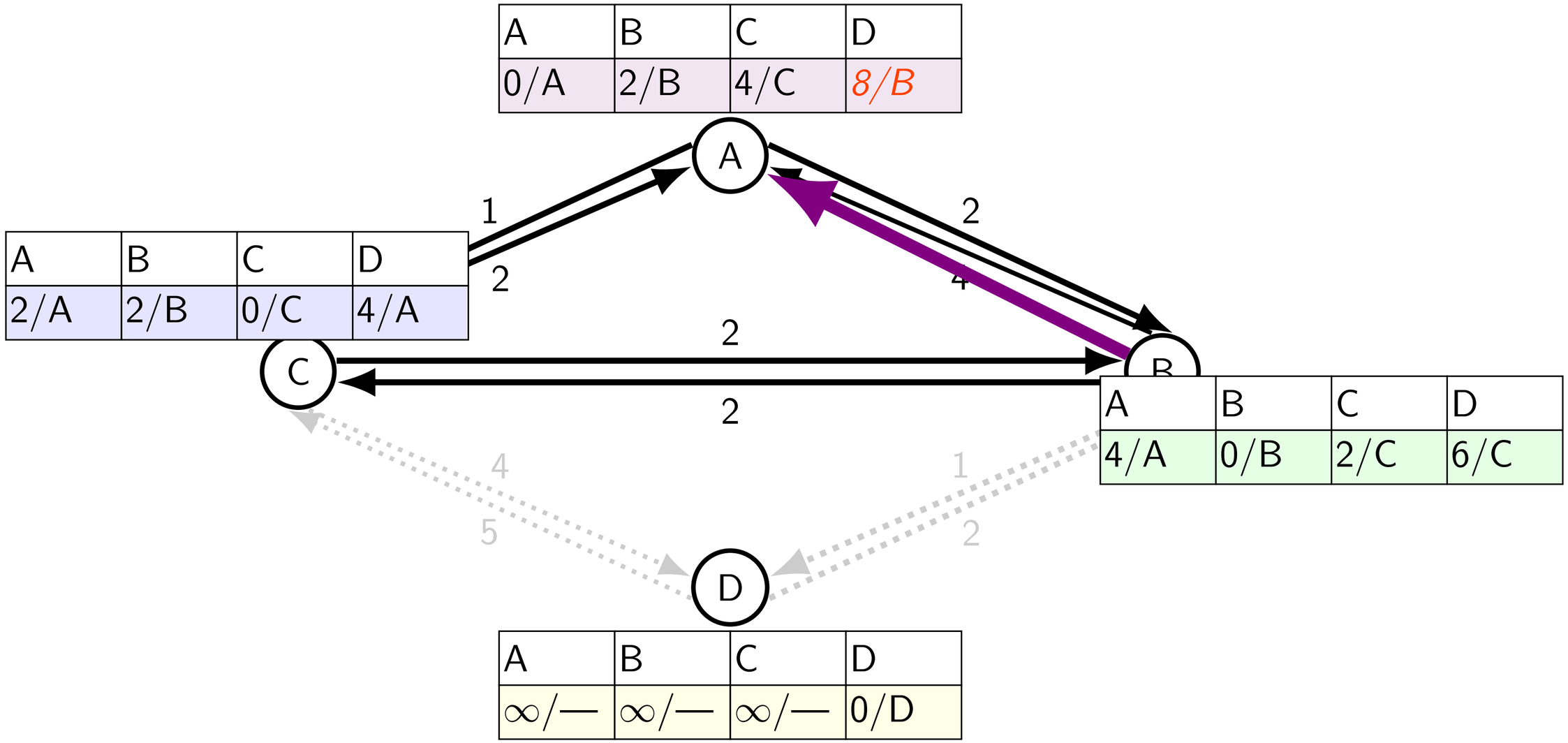
count-to-infinity (v2)



count-to-infinity (v2)



count-to-infinity (v2)



count-to-infinity

when node becomes unreachable, can have ‘phantom’ routes

keep propogating in loop, incrementing metric forever

RIP solution: maximum metric is 15 (hops)

better count-to-infinity solutions?

can share information about more than just neighbors

we'll see two examples:

link-state routing protocols (example: OSPF)

- every router learns full map of network

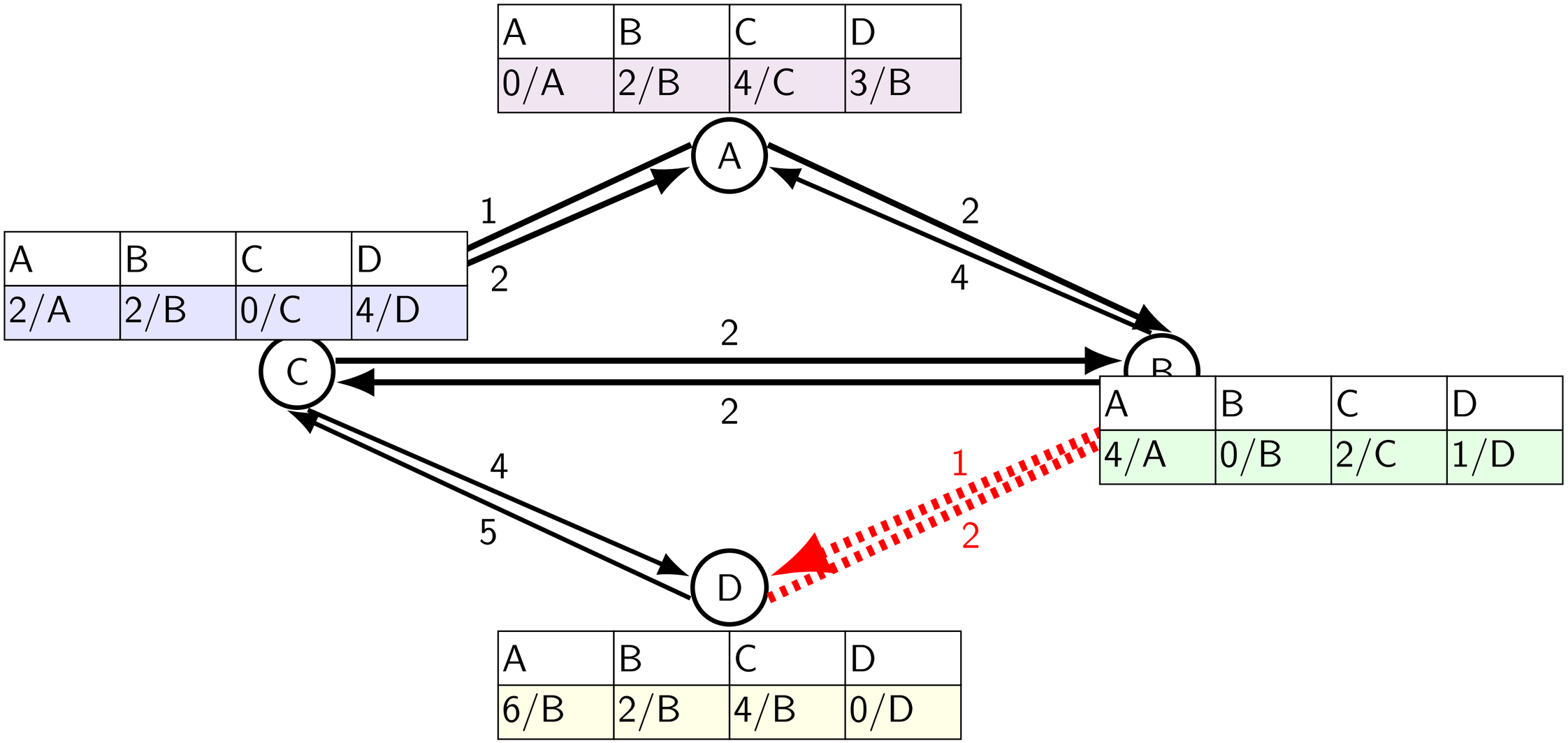
border gateway protocol (BGP)

- (basically) track *list of hops* alongside distances
- eliminate potential routes that would create duplicate hops (loops)

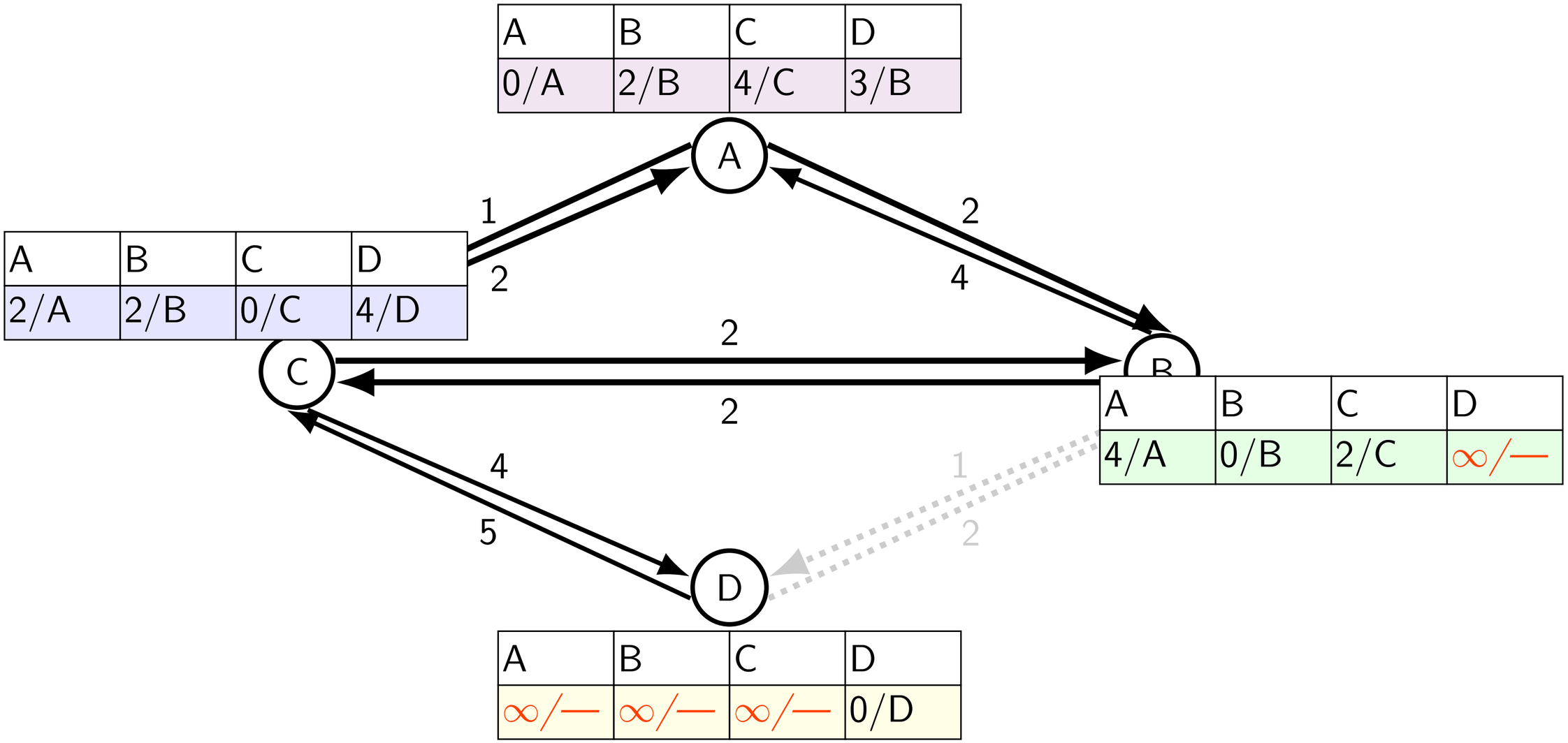
backup slides

count to infinity (no split horizon)

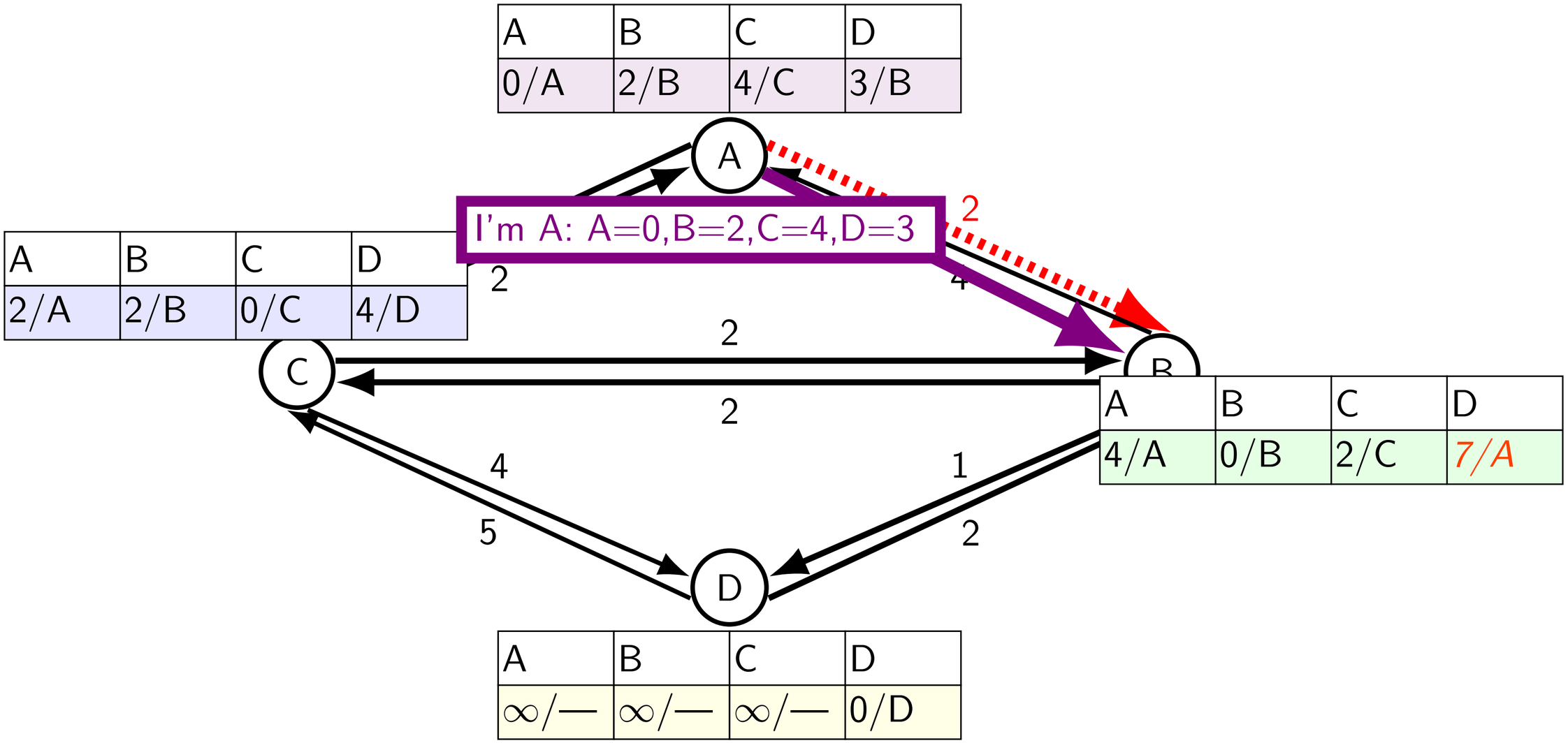
count to infinity (no split horizon)



count to infinity (no split horizon)



count to infinity (no split horizon)



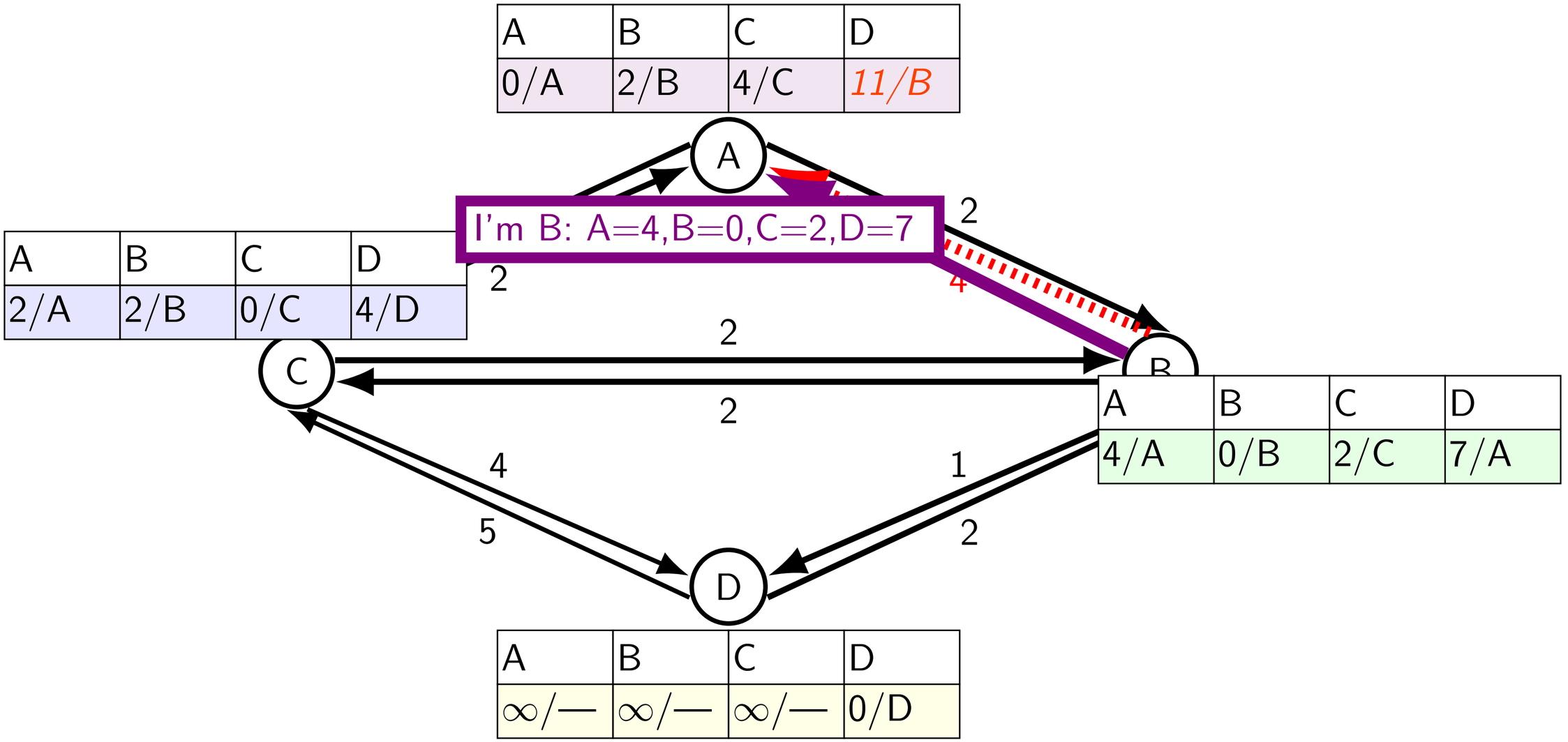
A	B	C	D
0/A	2/B	4/C	3/B

A	B	C	D
2/A	2/B	0/C	4/D

A	B	C	D
4/A	0/B	2/C	*7/A*

A	B	C	D
∞/—	∞/—	∞/—	0/D

count to infinity (no split horizon)



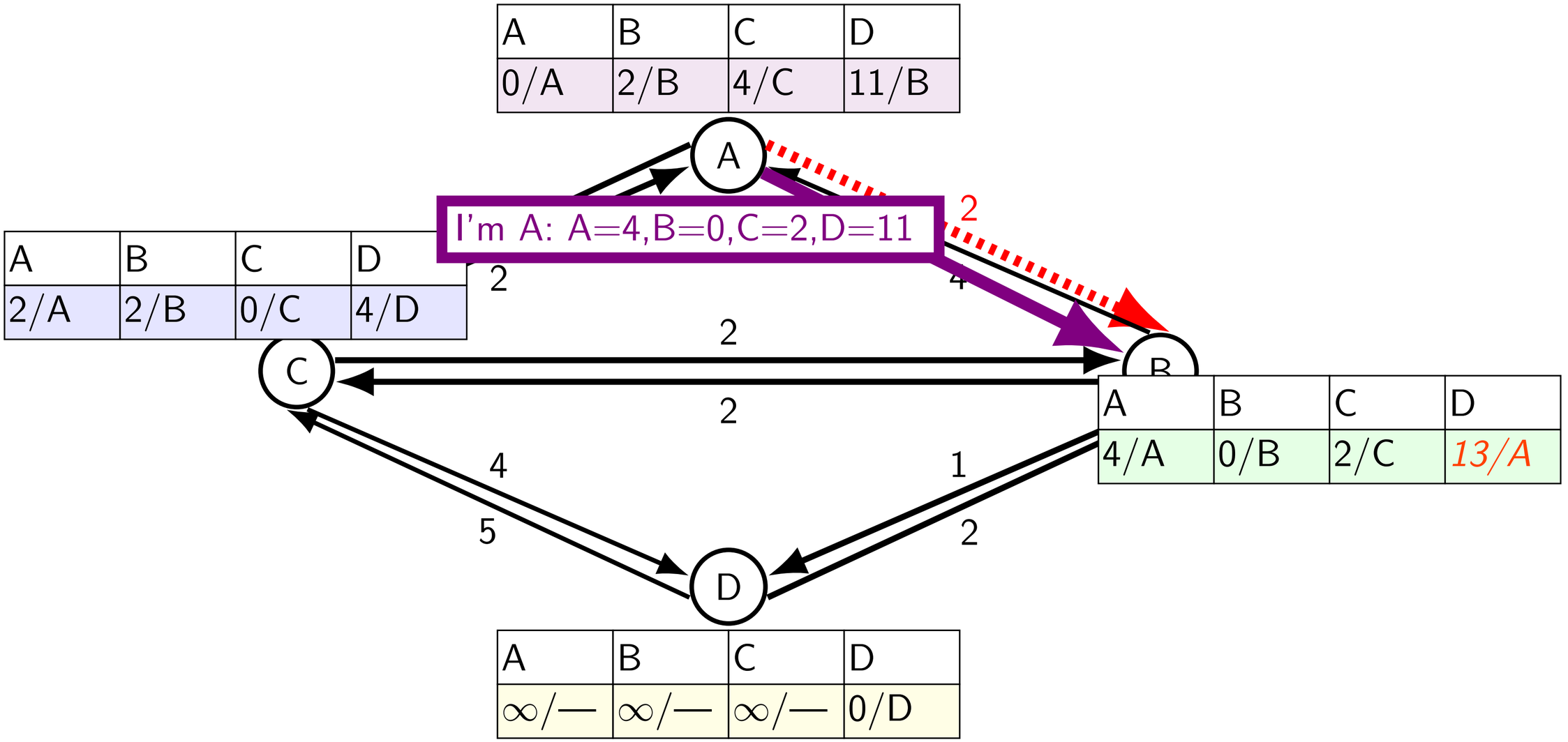
A	B	C	D
0/A	2/B	4/C	*11/B*

A	B	C	D
2/A	2/B	0/C	4/D

A	B	C	D
4/A	0/B	2/C	7/A

A	B	C	D
∞/—	∞/—	∞/—	0/D

count to infinity (no split horizon)



count to infinity (no split horizon)

